NeuroDSL: A Mutable Computational Graph Framework with Local Invalidation and Inspectable Memory Planning

Abdallah Khemais
ISITCOM, University of Sousse

Abstract

We present NeuroDSL, a deep learning framework built around a mutable directed acyclic graph (DAG). Unlike static frameworks, NeuroDSL keeps the computational graph as a persistent object that survives across training steps, so that architectural edits and parameter writes trigger localized recomputation rather than a rebuild of the whole graph. Our central claim is a pair of localities, and we measure both structurally — as exact node counts, deterministic and independent of any timer. *Invalidation locality*: on a 12-layer Transformer graph whose total size we inflate sevenfold with disjoint unreachable branches, the number of nodes an edit traverses does not change by a single unit, at any of the four sites measured (493, 452, 247 and 1 nodes at increasing depth), reproducing identically across independent process launches and matching an independently written BFS oracle on 16 of 16 points. Two controls bracket the measurement: a negative one that saturates the invalidatable set at 100%, and a positive one in which the padding is made reachable and the count grows by exactly the predicted 600 nodes per unit while every deeper site stays bit-identical — so the measure is shown both to respond to graph size and to ignore it where locality says it must. *Retrieval locality*, which to our knowledge has not previously been reported for a reactive graph engine and is not implied by the invalidation bound: evaluation walked the cached topological order from its start, so its cost was governed by the target's position in the whole graph rather than by what the target depends on. Restricting it to the target's real ancestor cone makes the retrieval cost invariant in total graph size — again 16 of 16 against an independent oracle — worth 12.59× and 2.96× on shallow and mid-depth targets in a graph holding independent subgraphs, and, we state as a condition rather than a caveat, exactly 1.00× on a graph holding one model, where there is nothing for the cone to exclude. The two localities run in opposite directions along depth — invalidation is cheap at deep sites, retrieval at shallow targets — so between them they cover the graph. Buffer allocation is planned by interval coloring, which by Dilworth's decomposition theorem uses the minimum number of buffer *slots* admissible for a given execution order; we report that this optimality is *vacuous* in the shipped configuration, because a graph expecting a backward pass has no two disjoint activation lifetimes, so the minimum equals the node count and no slot is ever shared (measured: 201 slots for 201 nodes on a 4-layer `LlamaModel`). A forward-only plan does share — 201 slots fall to 40 — yet the peak VRAM is unchanged at 46.96 MB, identical to unplanned execution, because the buffer pool retains what it reclaims; the mechanism that does lower the forward-only peak is eager release, at 16.96 MB (2.77× better). Both localities are consequences of one design decision — the graph keeps its activations resident — and so is its memory profile, which we report as the signature of that decision rather than as a separate result: on an end-to-end character-level language-model run at matched final validation loss, NeuroDSL uses 30% less peak VRAM than PyTorch on forward-only episodes and 25% more on the training step. It is about 2.2× slower per step, and throughput is not a contribution we claim. What the framework offers is mutability, locality and exactness. Implemented in pure Julia, it offers a declarative rule DSL for reactive graph composition, a node-fusion rewrite whose output we measure bit-identical to the unfused composition at every configuration tested, and an incremental backward pass that skips the part of the graph lying strictly upstream of the shallowest trainable parameter — 85.19% of the nodes receiving backward treatment on the frozen-prefix fine-tuning configuration we measure, against 33.33% for a wall-clock gain bounded at 4.19% when that prefix is absent. Its gradients agree with full backpropagation to within 10^{-6} on the topologies we tested, and are bit-identical (measured difference exactly 0) on the one we measure directly.

1 Introduction

Modern deep learning relies almost exclusively on static computational graphs [2, 3, 4]. While efficient, these frameworks force the user to define the whole network before training, making dynamic architectural changes, continual learning, and reactive behaviours cumbersome. Moreover, memory management in such frameworks is either left to the runtime [2] or requires manual checkpointing [3], often wasting precious GPU memory.

The design of NeuroDSL is partly inspired by the *JuliusGraph* engine of Julius Technology [1], which first demonstrated the potential of dynamic computational graphs. NeuroDSL extends this idea with explicit memory planning, graph-level rewrites, a rule-based DSL, and an interactive live viewer. We introduce NeuroDSL, a framework that treats the computational graph as a **mutable entity** capable of evolving during training. Its main components are:

1. **Dynamic DAG with lazy invalidation** – nodes and rules can be added or removed at any time; only the strictly necessary parts of the graph are recomputed. We measure the traversal count directly while inflating the total graph size sevenfold and it does not move (Section 7.1).

2. **Locality on both halves of a reactive step** – invalidation is bounded by the edited node's downstream cone, and *retrieval* by the target's ancestor cone. The second was missing until now: evaluation scanned the cached topological order from its beginning, so its cost was governed by the target's position in the whole graph rather than by what the target depends on. Section 7.2 measures what restricting it to the real ancestor cone buys, and states the condition under which it buys nothing.

3. **Rule DSL** – a declarative mini-language for composing computation rules over named tensors, enabling clean separation of topology definition from execution.

4. **Inspectable memory planning via interval coloring** – a greedy interval-coloring pass that assigns tensors with disjoint lifetimes to a shared buffer slot, using the provably minimum *number* of slots for a given execution order [12]. The optimality is real and so is its emptiness on a training graph: with a backward pass to follow, no two activation lifetimes are disjoint, so the minimum is attained trivially at the node count and *no slot is shared*. Sharing requires a forward-only plan, and even then the measured VRAM peak is unchanged. Section 6 gives the measurements and names the mechanism that does reduce the forward-only peak, which is not this one.

5. **Graph-level rewrites** – operator fusion and an incremental backward pass, both applied in place on the live graph.

6. **GPU backend** – hand-written CUDA kernels for common operations (RMSNorm [15], SwiGLU [16], softmax) and a buffer pool that reduces allocations.

7. **Interactive visualization** – a viewer that replays forward (green) and backward (red) passes from a recorded execution trace, in the spirit of Netron [21] but driven by the live graph.

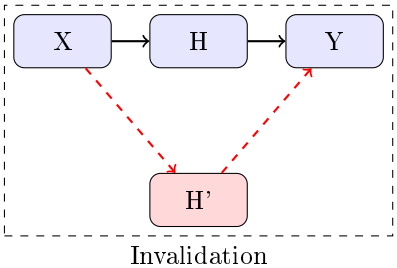

Figure 1: Lazy Invalidation Mechanism: when node H is replaced by H', all dependent nodes are marked invalid and recomputed on demand.

The architectural core of NeuroDSL is its reactive computational model, which enables localized updates without the overhead of global graph recompilation. As illustrated in fig. 2, unlike traditional frameworks that trigger a full rebuild upon topological changes, NeuroDSL confines invalidation to the immediate neighborhood of the modified parameter, ensuring efficient local evolution.

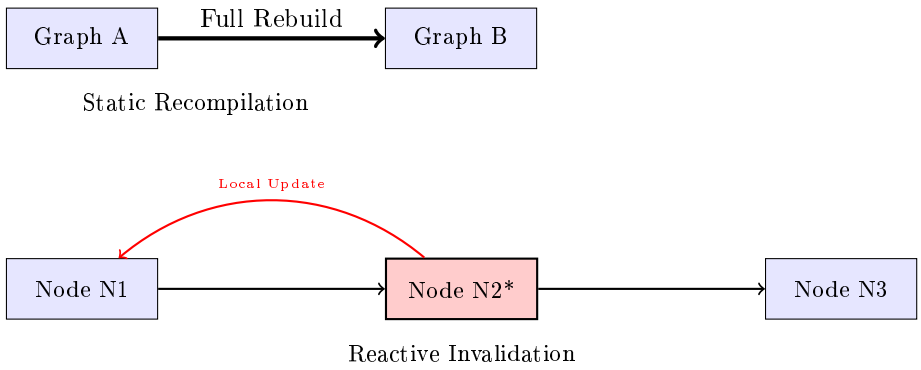

Figure 2: Comparison: Global static recompilation (top) vs. NeuroDSL reactive local invalidation (bottom).

We evaluate NeuroDSL against Flux and PyTorch on several benchmarks. The picture is mixed and we report it as such: on a single Transformer block NeuroDSL uses less peak GPU memory than PyTorch eager, while on an end-to-end language-model training run it is roughly 2.2× slower per step than PyTorch, and its peak memory on that run is 30% below PyTorch's on forward-only episodes and 25% above it on the training step (Section 7.8). Throughput is not where the framework is competitive, and neither, in general, is memory: what the persistent graph provides is locality, and the residency that makes locality possible is the same residency that makes a training step heavier. What the mutable graph changes is not the set of *expressible* training procedures — a static framework can always rebuild a model and copy weights across — but their cost and their continuity: an architectural edit becomes a local, in-place operation on a live graph that preserves optimizer state and cached activations outside the affected region, instead of a teardown-and-reconstruct step.

2 Background and Motivation

2.1 Computational Graph Paradigms

Deep learning frameworks can be broadly classified according to how they represent and execute computational graphs. Understanding these paradigms is essential to situating NeuroDSL's design choices.

Define-and-Run (Static graphs). Frameworks such as TensorFlow 1.x [4] and Theano require the user to declare the full computation graph *before* any data flows through it. The graph is then compiled into an optimised execution plan. This enables aggressive whole-graph optimisations (constant folding, dead-code elimination, layout transformations) but makes dynamic control flow (e.g., variable-length sequences, conditional layers) awkward to express.

Define-by-Run (Eager / Dynamic graphs). PyTorch [2] and Chainer [8] pioneered eager execution: the graph is built implicitly as operations are called, one Python statement at a time. This gives full Python expressiveness but means the graph is *recreated from scratch* on every forward pass – preventing reuse of graph structure across iterations and making incremental updates impossible.

Functional transformation pipelines. JAX [3] represents computations as pure functions and applies transformations (`jit`, `grad`, `vmap`) at the trace level. It achieves excellent performance through XLA compilation and is highly composable, but the purely functional model makes stateful, in-place graph mutations conceptually foreign.

Flux.jl. Flux [5] is Julia's mainstream deep learning library. It leverages Julia's multiple dispatch and the Zygote [6] automatic differentiation engine. Flux is elegant and fast for standard architectures, but like PyTorch it rebuilds the gradient tape on every call and offers no native mechanism for partially reusing backward computations.

NeuroDSL occupies a new position in this landscape: the graph is **persistent and mutable**, surviving across training steps, and changes to it trigger **surgical, lazy recomputation** of only the affected portions.

2.2 Memory Management in Deep Learning

Peak GPU memory is a dominant practical constraint in deep learning. State-of-the-art techniques include:

- **Gradient checkpointing** [9]: retains only a subset of activations during the forward pass; recomputes the rest during the backward pass, trading memory for computation.
- **ZeRO** [10]: shards optimizer states, gradients, and parameters across GPUs in a data-parallel setting.
- **Activation offloading / vDNN** [11]: moves tensors between GPU and CPU memory based on usage predictions.

Gradient checkpointing comes with an analytical memory/compute trade-off [9]; ZeRO and offloading are driven largely by heuristics or manual placement policies. NeuroDSL takes a different angle: rather than trading compute for memory, its interval-coloring planner exposes the buffer assignment itself as a first-class, inspectable object, and answers exactly one question optimally — the minimum *number* of buffer slots needed for a given execution order. This is a classical result for interval graphs rather than a new algorithm; the contribution is its availability inside a graph that mutates at runtime (Section 6).

3 Framework Comparison

Table 1 compares NeuroDSL's feature set with that of the major deep learning frameworks. Such tables are necessarily coarse and are written by the authors of one of the entries, so we have tried to be conservative in NeuroDSL's own column: a ✓ means the capability is exercised by the framework's test suite, and ∼ marks anything that exists only as a partial implementation or as a notebook-level experiment.

Table 1: Feature comparison across deep learning frameworks. ✓= native support; ∼ = partial / workaround / experiment outside the framework proper; × = not supported.

Feature	NeuroDSL	PyTorch	JAX	TF 2.x	Flux.jl	DyNet
Persistent mutable graph	✓	×	×	×	×	∼
Lazy invalidation	✓	×	×	×	×	×
Incremental backward pass	✓	×	×	×	×	×
Rule-based DSL	✓	×	×	×	×	×
Buffer-*count*-optimal memory plan, inspectable	✓	×	×	×	×	×
Automatic operator fusion	∼	✓	✓	✓	×	×
Reactive invalidation callbacks	✓	∼	×	∼	×	×
GPU-accelerated kernels	✓	✓	✓	✓	✓	∼
Multi-GPU support	∼	✓	✓	✓	∼	×
Live interactive visualizer	✓	×	×	∼	×	×
Pure Julia implementation	✓	×	×	×	✓	×
Dynamic architecture during train	✓	∼	×	×	×	✓

Three entries in NeuroDSL's column deserve their ∼ explicitly. **Operator fusion** is automatic in the sense that a pass scans the graph and applies matching rewrites, but only four fused operators currently have

a real execution branch; rewrite rules whose target operator has no kernel are detected and then refused by a dispatch guard, so the effective coverage is much narrower than the rule set suggests (Section 9). PyTorch is marked ✓ here because TorchInductor performs automatic fusion in PyTorch 2.x. **Multi-GPU** exists only as a notebook-level, two-device data-parallel experiment with periodic weight synchronisation (`notebook/MNIST_par.ipynb`); there is no multi-device support in the framework itself. **Memory planning** is optimal in the number of buffer slots, not in bytes, and on a graph expecting a backward pass that optimum is the node count, so no slot is shared and the peak is unchanged — the ✓ is for inspectability, not for a memory saving. See Section 6.2.

3.1 Detailed Comparison by Dimension

3.1.1 Graph Execution Model

PyTorch. PyTorch's `autograd` engine builds a fresh computation graph (the *tape*) on every forward call [2]. This means no information from the previous iteration is reused; the entire graph is discarded after `loss.backward()`. `torch.compile` (introduced in PyTorch 2.0) can cache and reuse compiled graphs, but only for *identical* input shapes and control flow — any structural change forces a full recompile.

JAX. JAX traces Python functions into an intermediate representation (Jaxpr) and compiles them with XLA [3]. The trace is cached for a given function signature, providing excellent repeat-execution performance. However, JAX's model is inherently functional: there is no persistent graph object to inspect or mutate; statefulness is modelled via explicit parameter dictionaries passed to and returned from each function call.

TensorFlow 2.x. TensorFlow 2.x defaults to eager execution (like PyTorch), with optional `@tf.function` decoration to trigger graph compilation. The resulting `ConcreteFunction` is reused for matching inputs but cannot be incrementally modified.

Flux.jl. Flux builds on Zygote's source-to-source AD, which generates the backward pass at compile time from Julia IR. The backward code is efficient but fixed; there is no mechanism to partially invalidate or partially reuse it.

NeuroDSL. NeuroDSL maintains a single `NeuroGraph` object across all training iterations. Structural mutations (node additions, rule swaps, parameter updates) are *surgically applied* to this persistent object. The lazy invalidation mechanism (Section 4.2) ensures that only the affected sub-graph is recomputed, while unchanged portions retain their cached values. This is conceptually closer to a reactive spreadsheet engine than to a tape-based AD system.

3.1.2 Memory Management

PyTorch. Memory is managed by the `THCCachingAllocator`, a CUDA caching allocator that avoids frequent `cudaMalloc` calls by maintaining a pool of free blocks. This reduces allocation overhead but does not minimise peak memory: the allocator cannot reason about the full tensor lifetime graph and may retain fragmented blocks unnecessarily. Gradient checkpointing [9] is available as a manual opt-in.

JAX. JAX relies on the XLA compiler to perform buffer assignment. XLA's liveness analysis and buffer reuse are sophisticated but happen at the XLA-HLO level, invisible to the user and non-introspectable at the framework level.

TensorFlow. TensorFlow's Grappler optimizer performs similar optimisations (memory swapping, recomputation) at the graph level, but they are heuristic and configuration-driven.

NeuroDSL. NeuroDSL's `MemoryPlan` performs an explicit interval-coloring of tensor lifetimes, as described in Section 6. By Dilworth's theorem, this yields the minimum number of buffer slots required for the execution order it was computed from. The plan is exposed as a first-class object, enabling users and tooling to inspect and reason about memory allocation decisions — which, rather than the slot count itself, is the difference from XLA's buffer assignment.

3.1.3 Operator Fusion

PyTorch / XLA / TVM. Operator fusion in mainstream frameworks is either compiler-driven (XLA, TVM [18]) or requires manual kernel authorship (PyTorch custom CUDA extensions). Compiler-driven fusion operates on low-level IR and is opaque to the user; custom kernels require significant engineering effort.

NeuroDSL. NeuroDSL exposes fusion as a **graph-level rewrite** on the live graph: a pattern-matching pass scans the rule set and replaces a matched chain with a single fused rule, without a separate compiler pipeline. The rewrite is transparent and extensible at the pattern level, but it is not a code generator: a pattern can only be applied if a kernel already exists for its target operator, and a dispatch guard refuses the rewrite otherwise. Coverage is therefore limited by the kernel library, not by the pattern language (Section 9).

3.1.4 Differentiation Strategy

Table 2: Differentiation strategies across frameworks.

Framework	AD Strategy	Backward reuse
PyTorch	Reverse-mode tape (dynamic)	None (tape rebuilt each step)
JAX	Source-to-source (functional)	Full reuse (XLA cache)
TF 2.x	GradientTape (eager) / XLA	Partial (`@tf.function`)
Flux.jl	Source-to-source via Zygote	None (recomputed each step)
DyNet	Reverse-mode tape (dynamic)	None
NeuroDSL	**Graph-persistent reverse-mode**	**Incremental (affected nodes only)**

4 Design of NeuroDSL

4.1 Mutable Computational Graph

The core of NeuroDSL is the `NeuroGraph` struct, which stores named tensors (`GraphNode`) and transformation rules (`GraphRule`) in per-namespace dictionaries, along with a cached topological order (`_topo_cache`). The graph is not a static trace but a live structure that can be modified at any point during training. When a node's value changes via `set!`, a dual invalidation sweep is triggered:

```
# Real implementation from graph_api.jl
function set!(g::NeuroGraph, name::Symbol, value;
              is_param=false, namespace=g.active_ns)
    _ensure_namespace!(g, namespace)
    value_on_device = Backend.to_device(g.device, value)
    # Preserve existing watchers and callbacks
    old_nd = get(g.nodes[namespace], name, nothing)
    g.nodes[namespace][name] = GraphNode(name, value_on_device;
        is_param=is_param, namespace=namespace)

    # 1. Forward sweep: clear valid flag on all downstream nodes
    _invalidate_downstream!(g, name, namespace)

    # 2. Backward sweep: wipe gradients of all ancestors (params only)
    if is_param
```

```
        _invalidate_upstream!(g, name, namespace)
    end
    return g
end
```

Listing 1: Core dynamic mechanism: set! triggers dual invalidation sweeps

The `demand!` function evaluates the graph in a pull-based fashion: it enumerates the *ancestor cone* of the requested target — the target itself plus every node it transitively depends on, in dependency order — and dispatches only the rules whose output node has `valid = false`, via the internal `execute_rule!` dispatcher. Nodes outside that cone are never examined, whatever their position in the graph. The cone is produced by `_ancestors_of!` (`src/graph_api.jl`) and cached per target, invalidated at the same points as the topological-order cache. Section 7.2 measures what this restriction costs and what it saves, against the alternative of scanning the cached topological order up to the target.

4.2 Lazy Invalidation Mechanism

The lazy invalidation mechanism is the cornerstone of NeuroDSL's efficiency for dynamic graphs. In conventional eager frameworks, every forward pass recomputes all nodes unconditionally. In NeuroDSL, nodes carry a `valid` flag. When a value is written via `set!`, two propagation sweeps are triggered:

1. **Forward invalidation** (`_invalidate_downstream!`): starting from the modified node, the algorithm performs a breadth-first traversal of the DAG's forward edges, clearing the `valid` flag on every reachable successor. These nodes will be recomputed the next time `demand!` is called.
2. **Backward invalidation** (`_invalidate_upstream!`): when a *parameter* is modified, its accumulated gradient is no longer meaningful. The sweep nullifies the gradients of the nodes computed using this parameter and, because it climbs from each consuming rule into that rule's other inputs, of those co-inputs as well — so it reaches strictly more than the forward-reachable set, deliberately, for the reason set out in Section 4.5.

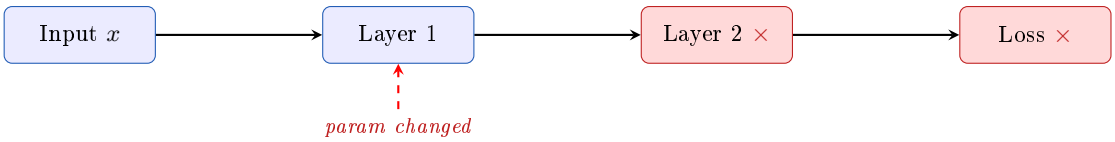

Figure 3: Lazy invalidation: modifying a parameter in Layer 1 marks only Layer 2 and Loss as invalid; Layer 1's unaffected predecessors retain their cached values.

The key algorithmic insight is that the topological order of the graph is cached and updated *only* when the graph's *structure* changes (nodes or rules added/removed). Value changes – the common case during training – never trigger a topological re-sort. This reduces the overhead of a typical training step to a linear scan of the cached order plus the actual kernel executions.

```
# Conceptual view of demand! evaluation
function demand!(g::NeuroGraph, target::Symbol;
                 namespace=g.active_ns)
    nd = g.nodes[namespace][target]
    nd.valid && nd.value !== nothing && return nd.value

    # _ancestors_of! returns the target's ancestor cone in dependency order,
    # cached per target; nodes outside the cone are never visited.
    for name in _ancestors_of!(g, namespace, target)
        nd_i = g.nodes[namespace][name]
        if !nd_i.valid && haskey(g.rules[namespace], name)
            # execute_rule! dispatches to the correct kernel via DISPATCH_TABLE
```

```
            # and stores forward-pass context for the backward pass
            execute_rule!(g, g.rules[namespace][name];
                          ctx_store=CtxStore(), namespace=namespace)
        end
    end
    return g.nodes[namespace][target].value
end
```

Listing 2: Lazy demand evaluation (simplified from graph_api.jl + dispatch.jl)

Note. In the real codebase, the per-operation dispatch happens in `dispatch.jl` through `_dispatch_op`, which resolves the `op::Symbol` stored in each `GraphRule` to the appropriate Julia kernel. The dispatch table approach (symbol → kernel) is intentional: it enables runtime rule replacement and operator fusion without redefining Julia closures. This pull-based, demand-driven evaluation is analogous to lazy evaluation in functional languages such as Haskell, but applied to a mutable, side-effecting GPU computation graph.

4.3 Rule DSL

A distinctive feature of NeuroDSL is its **Rule DSL** – a declarative mini-language for expressing computation rules over named graph nodes. Rather than writing imperative code that manipulates tensors directly, the user declares named rules that describe *what* to compute and *which* nodes are consumed and produced.

Motivation. In PyTorch or JAX, the computation is embedded in Python/Julia control flow and function calls. This conflates topology (which nodes depend on which) with computation (what the operation is), making it hard to inspect, modify, or optimize the graph at runtime. NeuroDSL's Rule DSL separates these concerns: topology is captured in the rule's `inputs` and `output` fields, while computation is referenced symbolically via the `op` field, looked up at dispatch time.

Rule anatomy. The `GraphRule` struct (from `types.jl`) has the following fields:

- `output::Symbol` – name of the `GraphNode` produced by this rule.
- `inputs::Vector{Symbol}` – ordered list of `GraphNode` names consumed.
- `op::Symbol` – symbolic operation name (e.g. `:matmul`, `:relu`, `:rmsnorm`), resolved to a kernel at dispatch time via `_dispatch_op`.
- `attrs::Dict{Symbol,Any}` – optional hyperparameters (e.g. `:trans_b`, `:eps`).
- `namespace::Symbol` – the namespace this rule belongs to.
- `atom_type::Type{<:NeurAtom}` – either `Datom` (data, no gradient) or `Quantom` (differentiable).

Backward rules are registered separately in the global `GRAD_RULES` dictionary (`backward.jl`), keyed by `op` symbol. This design means that adding a new operation requires only: (1) registering a forward kernel in `dispatch.jl`, (2) registering its gradient function in `GRAD_RULES`. No graph recompilation is needed.

```
g = NeuroGraph()

# Declare input and parameter nodes
set!(g, :x,  randn(Float32, 4, 64))
set!(g, :W1, randn(Float32, 64, 64) .* 0.1f0; is_param=true)
set!(g, :W2, randn(Float32, 64, 64) .* 0.1f0; is_param=true)
set!(g, :y,  zeros(Float32, 4, 64); atom_type=Datom)

# Stack rules to build a 2-layer MLP:
# h1 = relu(x * W1'), h2 = h1 * W2', loss = mse(h2, y)
addrule!(g, GraphRule(:h1_pre, [:x, :W1], :matmul;
                      attrs=Dict(:trans_b => true)))
```

```
addrule!(g, GraphRule(:h1,      [:h1_pre],       :relu))
addrule!(g, GraphRule(:h2,      [:h1, :W2],      :matmul;
                      attrs=Dict(:trans_b => true)))
addrule!(g, GraphRule(:loss,    [:h2, :y],       :mse_loss))

# Evaluate: only recomputes the nodes marked invalid
demand!(g, :loss)
```

Listing 3: Rule DSL usage. A rule names its inputs, its output, and a symbolic operation; it never carries a closure.

Namespaces. Rules and nodes can be grouped into *namespaces*, enabling modular composition of subgraphs. A Transformer block, for instance, can be defined in its own namespace and instantiated multiple times with different parameter names, without name clashes.

```
# Define an attention score/probability pair in namespace :attn1
addrule!(g, GraphRule(:scores, [:q, :k], :matmul;
                      attrs=Dict(:trans_b => true), namespace=:attn1))
addrule!(g, GraphRule(:probs,  [:scores],  :softmax, namespace=:attn1))

# The same rule names can be reused in a second namespace
addrule!(g, GraphRule(:scores, [:q2, :k2], :matmul;
                      attrs=Dict(:trans_b => true), namespace=:attn2))
```

Listing 4: Namespace-based modular composition

Comparison with other DSLs. TVM's Relay [18] and ONNX both define graph IRs, but they target compilation and export rather than interactive, mutable training. NeuroDSL's Rule DSL is designed to be *edited at runtime*: rules can be swapped, removed, or replaced during training (e.g., to implement neural architecture search or curriculum learning) and the invalidation system propagates the change automatically.

Declarative backward rules. Because gradients are looked up by operator symbol in `GRAD_RULES` rather than derived from traced code, the backward pass is declarative in the same sense the forward pass is. The consequence is a hard constraint rather than a convenience: NeuroDSL has no general automatic-differentiation fallback, so an operator with no entry in `GRAD_RULES` simply cannot be differentiated, and the fusion pass refuses to introduce one during training. Every operator used in the experiments of Section 7 has a hand-written gradient rule.

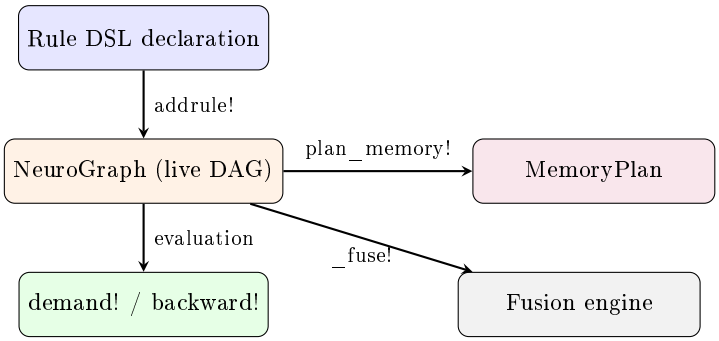

Figure 4: NeuroDSL architecture: the Rule DSL populates the live DAG, which drives evaluation, memory planning, and fusion.

4.4 Incremental Backward and Reactive Graph

To reduce backward computation when some parameters are frozen (fine-tuning, transfer learning, partial architecture updates), NeuroDSL exposes an opt-in `prune_frozen` flag on `backward_graph!` and `backward_graph_sparse!`. In a single forward topological pass, it computes which nodes have at least one trainable (`is_param=true`) ancestor; the reverse pass then skips gradient computation and propagation entirely for any node without one, rather than merely avoiding its storage.

The predicate is the one implemented in `_compute_needs_bwd` (`src/backward.jl`): a node needs backward treatment if and only if it is itself a trainable parameter, or at least one of its inputs needs it. Two consequences follow directly from that definition and both are used in Section 7. First, a *frozen* parameter is nothing more than a leaf with `is_param=false`; there is no separate frozen flag. Second — and this is the property that determines how much the mechanism can ever save — the set of skipped nodes is exactly the part of the graph lying strictly upstream of the shallowest trainable parameter. How much work `prune_frozen` removes is therefore governed by *where* the trainable parameters sit, not by how many parameters are frozen.

Gradient values for trainable parameters are unaffected: the framework's test suite compares the pruned and unpruned code paths on identical inputs and requires agreement within 10^{-6} on sequential and branched topologies, and where we have measured the difference directly it is exactly zero (Table 13). The flag defaults to `false`, so existing call sites are unaffected unless a caller opts in.

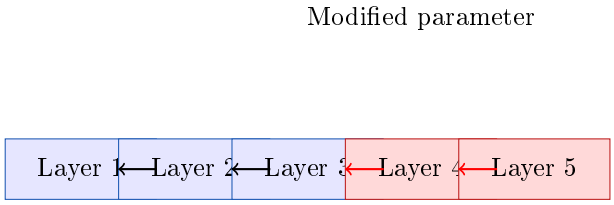

Figure 5: Incremental backward: only layers affected by the changed parameter (red) are recomputed.

Why this matters. In standard frameworks, `backward()` always differentiates through the entire computation graph. In many practical scenarios – fine-tuning (only the last few layers are updated), hyperparameter sweeps (a single regularization coefficient changes), or architecture search (one block is mutated) – only a small fraction of the gradients are actually affected. NeuroDSL's incremental backward directly exploits this structure, skipping the unaffected portions of the backward DAG.

Reactive callbacks. NeuroDSL extends the reactive model with `_watch!`, which registers one node as an observer of another: when the observed node is invalidated, the observer is invalidated too, and the observer's `on_change` callback — if it has one — fires. The mechanism is purely structural. It carries no value predicate and no threshold, so behaviour such as early stopping is not expressed by the callback itself but by user code that reads the node after a `demand!` and decides what to do; we make no claim about iteration savings from this mechanism.

```
# :probe observes :loss -- invalidating :loss also invalidates :probe
_watch!(g, :probe, :loss)
node(g, :probe).on_change = (graph, sym, ns) -> begin
    @info "node $sym was invalidated in namespace $ns"
end
```

Listing 5: Registering an invalidation observer via _watch!

4.5 Formal Proof of Invalidation Complexity and Impact Subgraph

A potential concern in dynamic frameworks handling very large DAGs ($|V| \to \infty$) is the computational overhead on the CPU during the graph traversal phase of `_invalidate_upstream!`. We provide a formal

analysis that bounds the complexity by the size of the *impact subgraph* — the set of nodes whose values depend on the modified parameter — rather than by the total graph size.

Impact subgraph. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the computational DAG. For a modified parameter θ, define the **impact subgraph** $\mathcal{G}_\theta = (\mathcal{V}_\theta, \mathcal{E}_\theta)$ as the subgraph induced by all nodes $v \in \mathcal{V}$ for which there exists a directed path from θ to v in $\mathcal{G}$. Equivalently, $\mathcal{V}_\theta$ is the set of nodes transitively reachable from θ.

Traversal correctness, and which sweep the bound covers. The bound is a statement about the *value* sweep, `_invalidate_downstream!`. That algorithm performs a breadth-first traversal from the edited node along the forward edges, clearing the `valid` flag on each node it reaches, and it enqueues a successor only through the consumers index. Its visited set is therefore the forward-reachable set from θ — which is $\mathcal{V}_\theta$ by the definition above, not merely a superset of it. On a graph whose nodes are all valid the number of visited nodes is exactly $|\mathcal{V}_\theta|$ and the total work is $\mathcal{O}(|\mathcal{V}_\theta| + |\mathcal{E}_\theta|)$. This is the function instrumented in Section 7.1, and because $\mathcal{V}_\theta$ is *defined* as forward reachability, the count measured there is the quantity this bound is about rather than a proxy for it.

$|\mathcal{V}_\theta|$ is an upper bound the engine can beat. The equality above is stated for a fully-valid graph, and the qualification is not idle. The traversal enqueues a successor only if that successor is *currently valid* (the `out_nd.valid` guard in `src/graph_api.jl`), because marking an already-invalid node is a no-op and everything downstream of it is invalid already, by induction on the same argument. So when part of the cone has been invalidated by an earlier edit and not yet recomputed, the traversal stops at the boundary of the still-valid region and visits *strictly fewer* than $|\mathcal{V}_\theta|$ nodes. The operational cost is therefore bounded above by $|\mathcal{V}_\theta|$ rather than equal to it, and the gap widens exactly in the regime a reactive engine is built for: successive edits before a demand, where later edits pay less because earlier ones already cleared part of their cone. We measure the fully-valid case throughout Section 7.1, which is the conservative choice — it is the worst case for this bound — and we note the slack rather than claim it, having not measured a partially-invalid graph.

The gradient sweep is not covered, and visits a superset. The second sweep, `_invalidate_upstream!`, must not be read as satisfying the same bound, and we state why explicitly rather than leaving the two functions to be conflated. It is not a reverse traversal of $\mathcal{E}^T$. From each dequeued node it performs two steps: a **JUMP**, enqueueing every rule output that consumes the node, and then a **CLIMB**, enqueueing every input of that rule. The climb is what breaks the bound. Reaching a downstream rule's output brings in *all* of that rule's inputs, including co-inputs that are not forward-reachable from θ — a sibling operand, a weight matrix belonging to another parameter, an unrelated branch feeding the same operator. Its visited set is therefore a *strict* superset of $\mathcal{V}_\theta$ in any graph with a rule of arity greater than one, which is every graph of interest. That behaviour is intended, because a gradient that was accumulated using a co-input is stale once the co-input's rule output changes, and nullifying it is what keeps the backward pass correct. But its cost is not $\mathcal{O}(|\mathcal{V}_\theta|)$, we do not claim it is, and Section 7.1 does not measure it. What we claim for the gradient sweep is correctness, established by the exactness results of Table 13, and not locality.

Bounds in common architectures.

- **Networks without skip connections (e.g., simple MLPs).** If θ belongs to layer k, the impact subgraph consists of nodes in layers $k, k+1, \ldots, K$ (the output layer). Its size is $\mathcal{O}((K-k) \times W)$, where W is the width of a layer. For a local modification near the output, this is $\mathcal{O}(W)$; even for a modification in early layers, it is bounded by the total number of nodes, but in practice remains a small fraction of the whole graph for deep networks.
- **Architectures with skip connections (ResNets, Transformers).** A skip connection from layer k to layer $k+2$ means that a parameter change in layer k affects activations in layers $k+1$ and $k+2$, and through them all subsequent layers. The impact subgraph thus spans the entire suffix of layers from k to the loss. Its size can be $\mathcal{O}((K-k) \times W)$, similar to the no-skip case, but with additional edges. Crucially, the invalidation *does not stop* at namespace boundaries: the algorithm follows all dependency edges, including those arising from residual connections. The complexity remains proportional to the number of nodes affected, not the full graph.

Why the bound is still practical. Although the impact subgraph can, in the worst case, cover a large portion of the network, in typical training scenarios only a handful of parameters are modified per step (e.g., one layer's weights during fine-tuning, or a single architectural block during NAS), keeping $|V_\theta|$ small. That said, the traversal cost is not the whole story: as our GPT-2-shaped experiment shows (Section 7.7), a small $|V_\theta|$ does not automatically translate into a faster wall-clock backward pass once implementation overhead is accounted for.

What is measured. Section 7.1 measures the traversal count of the value sweep directly, for the reason given above: that sweep's visited set is $\mathcal{V}_\theta$ itself. The independent verification of *which* nodes each sweep marks is a separate matter and is handled by the property tests (`test/test_property_invariants.jl`); what Section 7.1 adds is how the count behaves as $|\mathcal{V}|$ grows, which no test measures.

Summary. The original claim of $\mathcal{O}(M)$ with M the namespace size was too simplistic; it incorrectly assumed that inter-namespace edges could be pruned without affecting correctness. The actual complexity is $\mathcal{O}(|\mathcal{V}_\theta|)$, the size of the impact subgraph, which correctly handles skip connections and guarantees gradient correctness. This complexity is still bounded by the number of nodes in the affected suffix of the network, and is entirely independent of the total depth K in the sense that nodes not influenced by θ are never visited.

4.6 Automatic Operator Fusion

NeuroDSL can rewrite chains of rules into a single fused rule, removing one intermediate node and its dispatch. Two entry points exist. The legacy one, `_fuse!(g, chain)`, is triggered explicitly by the user and consults a two-entry pattern table (`FUSION_TABLE`); of its two entries, only `matmul+relu` has a kernel, so `relu+matmul` is declared but not executable. The current one is a scanning pass that matches rewrite rules against the graph and applies them itself. Both are gated by the same dispatch guard: a rewrite whose target operator has no execution branch, or no gradient rule while training, is refused rather than applied. Only four fused operators pass that guard today. Adding patterns is cheap; adding the kernels they need is not, and that — not the pattern language — is what bounds coverage (Section 9).

Mechanism. When a chain is matched, the intermediate rules are replaced by a single `GraphRule` whose `op` symbol names the fused kernel. The fused rule inherits the inputs of the first rule and the output of the last rule, and all references to the intermediate nodes are redirected. The rewrite requires the chain to be strictly linear — every intermediate node must have exactly one consumer, namely its successor in the chain — so that no external node observes a value the fused rule no longer materializes. It is performed in place on the live `NeuroGraph` and is immediately reflected in subsequent `demand!` calls.

```
# Before fusion: two separate rules
addrule!(g, :h,   inputs=[:W,:x], fn=(W,x)->W*x)
addrule!(g, :act, inputs=[:h],    fn=x->relu.(x))

# After fusion: single fused kernel
_fuse!(g, [:h, :act])
# g now contains a single :act rule backed by fused_matmul_relu kernel
```

Listing 6: Operator fusion API

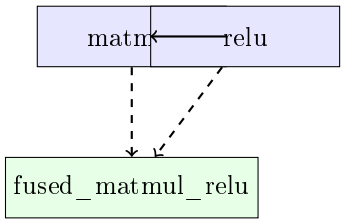

Figure 6: Operator fusion: two kernels replaced by a single fused kernel.

5 Mathematical Formalization and Core Theorems

The formal analysis of the invalidation complexity has been revised and moved to Section 4.5. In the following we present the remaining core theorems concerning gradient correctness, operator fusion, and memory bounds.

5.1 Consistency and Exactness of Reactive Automatic Differentiation

The pull-based evaluation scheme poses a fundamental question regarding the numerical accuracy of gradient computations when certain branches of the graph are not re-evaluated following partial invalidations.

Definition 1 (Partially Invalid Graph)**.** *A reactive computational graph is said to be consistent at a steady state if, for every node $v \in \mathcal{V}$ marked as valid (`valid = true`), its cached value matches exactly the global functional evaluation applied to its immediate parents: $\mathcal{V}_v = f_v\left(\{\mathcal{V}_u\}_{u \in Parents(v)}\right)$.*

Theorem 1 (Exactness of Reactive Gradients)**.** *Let $\mathcal{G}$ be a NeuroDSL execution graph in a partially invalid state. Assuming that structural mutations (node additions or topological rewrites) are strictly frozen between a forward execution pass and its corresponding backward derivative sweep, the pull-based evaluation triggered by a call to `backward_graph!` guarantees that for any parameter θ, the accumulated gradient matches exactly the full analytical gradient derived via the classical chain rule:*

$$\nabla_\theta \mathcal{L} = \sum_{p \in Paths(\theta \to \mathcal{L})} \prod_{(u,v) \in p} \frac{\partial f_v}{\partial \mathcal{V}_u} \tag{1}$$

The theorem is a statement about *gradients only*. It says nothing about optimizer state: a stateful optimizer such as AdamW keeps its own moment estimates (m_t, v_t) outside the graph, and whether those should be updated for a parameter whose gradient was recomputed rather than freshly invalidated is a design question, not a corollary of the theorem. We do not settle it here, and the experiments in Section 7 use a conventional optimizer that updates all moments unconditionally.

Proof. We proceed via backward mathematical induction on the topologically ordered structure of the DAG, moving from the objective loss node $\mathcal{L}$ down to the target parameter θ.

Base Case. For the terminal node $\mathcal{L}$, the seed value for backpropagation is $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = I$. The base condition holds immediately.

Inductive Step. Assume the exactness property holds for all immediate successors of an intermediate node v. When a reactive backward sweep is initiated by `backward_graph!`, the engine queries the upstream subgraph using the `demand!` mechanism. Under the temporal topological invariance assumption, no structural modifications occur during this step. Two configurations can arise for each successor $s \in \text{Succ}(v)$:

1. If node s is marked valid (`valid = true`), its cached value and the local Jacobians $\frac{\partial f_s}{\partial \mathcal{V}_v}$ stored within the evaluation context are mathematically exact, as they have not been altered by upstream modifications.
2. If node s is invalid (`valid = false`), the internal call to `demand!(g, s)` synchronously forces the functional re-evaluation of s and its entire invalid ancestral lineage. The local `CtxStore` context block is refreshed instantly on the GPU, guaranteeing the mathematical accuracy of the partial Jacobian.

By applying the linearity of the differentiation operator, the gradient accumulation is expressed as:

$$\frac{\partial \mathcal{L}}{\partial \mathcal{V}_v} = \sum_{s \in \text{Succ}(v)} \frac{\partial \mathcal{L}}{\partial \mathcal{V}_s} \cdot \frac{\partial f_s}{\partial \mathcal{V}_v} \tag{2}$$

Since each term of the summation is exact by either the inductive hypothesis or the synchronous on-demand re-evaluation, the gradient accumulated at θ equals the chain-rule gradient. Note that the argument is exact in exact arithmetic: in floating point, a recomputed node and a cached node may differ in the last bits, so what is established is that partial invalidation introduces no *systematic* error, not bit-identity. Bit-identity, where it holds, is an empirical fact about a particular graph and not a corollary of the theorem — we measure it on one topology in Table 13, alongside the tolerance the test suite asserts in general. □

5.2 Semantic Preservation of Operator Fusion Transformations

Automated operator fusion within NeuroDSL is formally defined as a structural graph rewrite that preserves the algebraic integrity of the model.

Definition 2 (Linear Fusion Homomorphism)**.** *An operator fusion transformation is a graph homomorphism $\mathcal{F} : \mathcal{G} \to \mathcal{G}'$ that contracts a purely sequential, linear chain of primitive operator nodes $\mathcal{C} = \{v_1, v_2, \ldots, v_n\}$ into a single execution vertex v_{fused}, while leaving the remainder of the graph $\mathcal{G} \setminus \mathcal{C}$ invariant.*

Theorem 2 (Semantic Preservation and Acyclicity)**.** *Let $\mathcal{F}$ be the fusion transformation applied to a linear chain of primitive operators. The fused executable function satisfies numerical epsilon-equivalence ($\overset{\epsilon}{=}$) with respect to discrete operator composition:*

$$\mathcal{K}_{fused}(x) \overset{\epsilon}{=} f_n \circ f_{n-1} \circ \cdots \circ f_1(x) \tag{3}$$

where $\epsilon \geq 0$ characterizes the arithmetic drift induced by compiler-level intermediate register allocation (e.g., Fused Multiply-Add tracking) relative to discrete global VRAM memory truncation. If the linear poset boundary condition holds—namely, that no external node $u \notin \mathcal{C}$ possesses both an incoming edge from $\mathcal{C}$ and an outgoing edge pointing to $\mathcal{C}$—then the transformation $\mathcal{F}$ preserves global evaluation semantics up to ϵ and introduces no cycles.

Proof. To demonstrate that the transformed graph retains its status as a DAG, suppose by contradiction that the target graph $\mathcal{G}'$ contains a cycle passing through the newly created vertex v_{fused}. This implies the existence of a directed path in $\mathcal{G}'$ originating from v_{fused} and returning to it.

Since the fusion homomorphism contracts only the linear chain $\mathcal{C}$, this path must traverse at least one external vertex $u \in \mathcal{G} \setminus \mathcal{C}$ such that $(v_{\text{fused}}, u) \in \mathcal{E}'$ and $(u, v_{\text{fused}}) \in \mathcal{E}'$. Mapping this back to the original graph topology $\mathcal{G}$ implies the existence of the following direct interface connections:

$$\exists v_i \in \mathcal{C} \text{ s.t. } (v_i, u) \in \mathcal{E} \quad \text{and} \quad \exists v_j \in \mathcal{C} \text{ s.t. } (u, v_j) \in \mathcal{E} \tag{4}$$

If $i < j$, the original path $v_i \to u \to v_j$ would form a directed cycle in $\mathcal{G}$ when combined with the internal sequential path $v_j \to \cdots \to v_i$, contradicting the fact that the initial graph $\mathcal{G}$ is a valid DAG. If $i \geq j$, this directly violates the linear poset boundary condition stating that no external node can maintain a bidirectional dependency with the internal sequential chain. This establishes acyclicity. Semantic equivalence up to ϵ is the weaker of the two claims and does not follow from the same argument: it holds because the fused kernel computes the same composition of the same primitives, and ϵ bounds only the discrepancy from carrying intermediates in registers rather than rounding them to memory. Bit-identity is not claimed by the theorem. For the one fused operator we validated numerically, the measured maximum absolute difference against the unfused composition is in fact exactly 0 at every one of the 36 launches archived for it — all six (device, dimension) configurations, on both the current and the pre-fix engine (Table 14); we have not established a bound on ϵ analytically, so ϵ-equivalence should be read as an empirical property of the fused kernels we ship rather than a proved property of the rewrite in general, and the exactness we measure is a property of those configurations rather than a corollary of anything above. □

5.3 Optimal Upper Bounds via Dilworth's Poset Decomposition

Spatial optimization of GPU memory layout within NeuroDSL moves away from empirical allocation heuristics (such as best-fit or first-fit policies) by utilizing the structural theory of partially ordered sets.

Definition 3 (Liveness Poset)**.** *Let $\mathcal{T}$ denote the set of intermediate activation tensors generated during a single execution pass. We define a partial order relation $\prec$ on $\mathcal{T}$ such that:*

$$t_1 \prec t_2 \iff e_{t_1} < s_{t_2} \tag{5}$$

where s_t and e_t denote the allocation step and the final read step of the liveness interval of tensor t, respectively. Two tensors are said to be incomparable if and only if their liveness intervals overlap in the time domain: $t_1 \parallel t_2 \iff I_{t_1} \cap I_{t_2} \neq \emptyset$.

Theorem 3 (Optimal Slot Count)**.** *Fix an execution order and let* $(\mathcal{T}, \prec)$ *be the resulting Liveness Poset. The minimum number of buffer* slots *needed to schedule all tensors in* $\mathcal{T}$ *without a liveness collision equals the width of the maximum antichain of* $(\mathcal{T}, \prec)$*, and greedy interval coloring attains it.*

Three restrictions on this statement matter, and we state them before the proof rather than after it. The first is measured and is the one a reader should weigh: the theorem is silent on whether *any* antichain is narrower than $\mathcal{T}$ itself, and in this framework's default configuration none is — every activation is live to the end of the schedule, so the width equals $|\mathcal{T}|$ and the optimum is the node count, with no slot shared. Section 6.2 reports this (201 slots for 201 nodes) together with what a forward-only plan changes and what it does not. Second, the quantity minimized is a *count of slots*, not a number of bytes. When tensors have unequal sizes, a slot must be dimensioned for the largest tensor assigned to it, and a coloring that is optimal in slot count need not be optimal in total bytes; the antichain width is then a lower bound on buffer concurrency and nothing more. NeuroDSL does not solve the byte-optimal variant, which is a dynamic-storage-allocation problem and NP-hard in general. Third, the optimum is relative to *one* execution order: choosing the topological order that minimizes the width is a separate problem the planner does not address — it colors the order it is given.

Proof. By direct application of **Dilworth's Decomposition Theorem**, the size of the largest antichain in a finite partially ordered set is equal to the minimum number of chains needed to partition the set exhaustively.

In the defined Liveness Poset, an antichain represents a subset of mutually incomparable tensors, meaning geometrically that all tensors within this antichain have overlapping lifespans at a specific execution step on the GPU. Consequently, no two tensors in an antichain can share the same physical address space without data corruption. If $\mathcal{A}_{\max}$ denotes this maximal antichain, the physical runtime environment must provide at least $|\mathcal{A}_{\max}|$ concurrent buffers.

Conversely, Dilworth's theorem guarantees that the entire set $\mathcal{T}$ can be partitioned into exactly $|\mathcal{A}_{\max}|$ disjoint chains $\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_{|\mathcal{A}_{\max}|}$. By the definition of a chain, for any pair of elements $(t_a, t_b) \in \mathcal{C}_k$, we have either $t_a \prec t_b$ or $t_b \prec t_a$, which implies that their liveness intervals are disjoint in time. The greedy interval-coloring allocator realizes this partition directly: scanning the execution order and reusing the lowest slot freed by an expired interval assigns one slot per chain. The two bounds meet, so the slot count is optimal. □

Relation to classical register allocation. The problem of assigning non-overlapping intervals to a minimum number of physical buffers is known in compiler design as *register allocation by graph coloring*, and has been solved optimally for interval graphs since the 1980s. Neither the algorithm nor the appeal to Dilworth's theorem is new here, and the theorem above should be read as situating a classical result rather than establishing one. What NeuroDSL adds is where the result is applied: the liveness intervals are recomputed as the graph evolves, and the resulting assignment is a first-class object the user can inspect. We note for completeness that the memory measurements reported in Section 7.3 were obtained on the default execution path, which does not route allocations through this planner; they therefore reflect the engine's ordinary buffer reuse and gradient-accumulation behaviour, not the interval-coloring pass.

6 Formal Memory Optimization Framework

NeuroDSL's `MemoryPlan` analyses the lifetime of every tensor in the graph and assigns buffer slots using greedy interval colouring [12]. By Dilworth's theorem, this colouring uses the minimum possible number of *slots* for the execution order it was computed from, equal to the width of the corresponding liveness order. Given the slot assignment and the tensor sizes, the peak footprint of the planned region follows by construction and can be read off before execution — which is what `MemoryPlan` makes available and what heuristic methods such as gradient checkpointing [9] or vDNN [11] do not. This is a weaker statement than minimising peak memory, and we do not claim the latter.

6.1 Liveness Analysis and Interval Coloring

To establish optimal allocation, NeuroDSL formalizes the lifetimes of tensors generated throughout a topological execution path. Let $G = (V, E)$ represent the computational Directed Acyclic Graph (DAG), where V is the set of nodes (tensors) and E corresponds to operational dependencies.

1. **Liveness interval formulation:** For each node $v \in V$, its live interval $I_v = [t_{\text{first}}(v), t_{\text{last}}(v)]$ is calculated. The generation step $t_{\text{first}}(v)$ is defined by the node's topological index during scheduling:

$$t_{\text{first}}(v) = \text{topo\_index}(v) \tag{6}$$

The eviction boundary $t_{\text{last}}(v)$ signifies the final step at which v is actively consumed by any successor rule:

$$t_{\text{last}}(v) = \max_{u \in \text{Succ}(v)} \Big(\text{topo\_index}(u)\Big) \tag{7}$$

2. **Interval Graph Construction:** We derive an interval graph $G_{\text{int}} = (V, E_{\text{int}})$ from these boundaries, where an edge exists between two nodes if and only if their intervals intersect:

$$E_{\text{int}} = \Big\{(u, v) \in V \times V \;\Big|\; I_u \cap I_v \neq \emptyset\Big\} \tag{8}$$

3. **Optimal Greedy Allocation via Dilworth's Theorem:** Because interval graphs are characteristically perfect, the chromatic number $\chi(G_{\text{int}})$ matches the size of its maximum clique $\omega(G_{\text{int}})$. Invoking Dilworth's theorem on the interval order, an earliest-deadline-first (EDF) scan assigns buffer colors $\mathcal{C}(v) \in \mathbb{N}$ such that:

$$(u, v) \in E_{\text{int}} \implies \mathcal{C}(u) \neq \mathcal{C}(v) \tag{9}$$

The number of slots the scan allocates therefore matches the maximum number k^* of simultaneously live tensors:

$$\text{Slots allocated} = \omega(G_{\text{int}}) = k^* \tag{10}$$

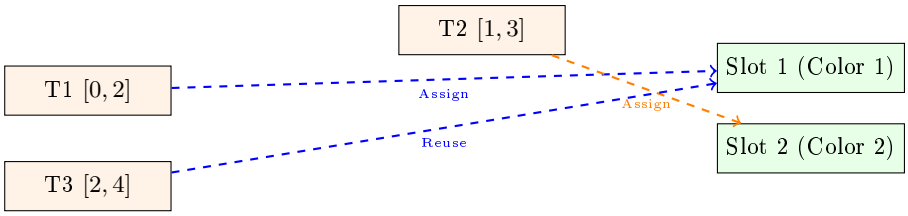

Figure 7: Interval Coloring Pipeline: Non-overlapping nodes (T1 and T3) safely share the same physical memory buffer slot via graph-level coloring transformations.

The plan is computed by `plan_memory!(g)` and returns a `MemoryPlan` object that maps each node to a buffer slot, together with a `BufferPool` that reuses those slots at execution time via `demand_planned!`. This is an opt-in execution path: the default `demand!` does not consult a plan.

6.2 What the Planner Actually Delivers

The theorem above is true and, in the configuration this framework ships, it is also empty. We measured it rather than assuming it, and the result changes what may be claimed.

The optimum is attained trivially, and nothing is shared. The slot count is a structural quantity — deterministic, no timer — so it can simply be counted. On every graph tested, `n_slots` equalled the node count exactly: 25/25, 61/61, 121/121, and **201/201** on a 4-layer `LlamaModel`. The planner shared *zero* slots. The cause is not a coloring defect but a liveness definition: `compute_liveness` (`src/liveness.jl`) extended `last_use` to the end of the schedule for every backpropable activation *unconditionally*, without asking whether a backward pass would follow. Every activation was therefore live until the last step, no two intervals were disjoint, and the minimum number of slots was the number of nodes. Dilworth's bound was attained — it had nowhere else to go.

Interval coloring is optimal in slot count for a fixed execution order; on a graph expecting a backward pass, that optimum is the node count, and optimality without disjointness buys nothing.

We are explicit that the earlier phrasing — "assigns tensors with disjoint lifetimes to a shared buffer slot" — invited a reader to infer that sharing occurs. In the shipped configuration it never did, and by construction it cannot for any training graph.

Sharing, once a forward-only plan is available. Placing the lifetime extension behind a `for_backward` keyword, defaulting to `true` so no existing behaviour moves, makes disjointness possible when the caller knows no backward will follow. The reduction is then substantial and was pre-registered as a go/no-go: abandon if the forward-only count reached $0.9n$, proceed if it fell below $0.6n$.

Table 3: Buffer slots against node count, structural and timer-free. `for_backward=true` is the shipped default. Artifact: `notebook/bench_liveness_slots_results.txt`.

Graph	Nodes	Slots, backward	Slots, forward-only	Fraction
8-layer chain	25	25 (no sharing)	10	0.400
20-layer chain	61	61 (no sharing)	22	0.361
40-layer chain	121	121 (no sharing)	42	0.347
4-layer `LlamaModel`	201	**201 (no sharing)**	**40**	**0.199**

The same artifact carries a positive control, in the sense of Section 7.1: adding one node to the graph moves the slot count by one (61 nodes/22 slots → 62/23), so the measure responds rather than merely reporting a constant.

Two further defects in the planned execution path, both found by measuring it. Correctness first: `demand_planned!` produces output bit-identical to `demand!` on every arm reported here. Getting there required two fixes beyond the liveness one, and each had made the path useless in a way no single-call test could reveal. First, the routine guessed a rule's output shape as its first input's shape, so `execute_rule!` freed each freshly acquired pooled buffer and immediately reallocated — **pool reuse was 0%** (0 hits against 164 allocations). It now calls `_infer_output_shape`. Second, it released raw *input leaves*, which no rule can recompute, so the path worked exactly once and failed on the second call with a type error where a released value should have been. After both fixes, the forward-only arm reaches a **92.1% pool hit rate** (151 hits, 13 allocations) at 40 slots, against 0.0% and 201 slots on the default path. The full test suite (2859 tests) passes after each fix. Artifact: `notebook/bench_planned_exec_results.txt`.

And it saves no memory. This is the part that matters. Slot count is not bytes, and pooling is not freeing. We measured the absolute VRAM peak of the three execution paths on the same forward-only char-LM workload used in Section 7.8 (one arm per process, each arm's output checked bit-identical against `demand!` in its own process).

Table 4: Absolute peak VRAM, forward-only, char-LM (vocab 65, dim 256, 4 heads, hidden 512, 4 layers, block 256), CUDA pool watermark, one arm per process, all arms bit-exact against `demand!`. Artifact: `notebook/bench_planned_vram_results.txt`.

Execution path	Peak	Notes
`demand!` (default)	46.96 MB	no plan consulted
`demand_release!`	**16.96 MB**	2.77× **lower**; calls `Backend.free!`
`demand_planned!`	46.96 MB	43 slots, 1169 pool hits — *identical to no plan*

The planned path's peak is identical to the unplanned one, to the reported precision, despite sharing slots and hitting the pool 1169 times. The reason is mechanical: `release!` returns a buffer to a pool bucket and never releases it to the device, so the watermark — which is what a peak measures — never falls. Slot

sharing reduces how many distinct buffers the plan *names*; it does not reduce how many bytes the process holds. The mechanism that reduces the forward-only peak is eager release, `demand_release!`, which calls `Backend.free!` and is 2.77× better than either alternative.

What we therefore claim. That the buffer assignment is an *inspectable object* on a graph that can still change — which is what the paper's title says and what the planner genuinely provides. That the slot count is optimal for a fixed order, with the qualification that on a training graph this optimum is the node count. That a forward-only plan reduces slots by a measured factor of 2.5–5. And that none of this reduces peak memory, for which the answer in this framework is `demand_release!`. We do not claim the planner saves memory, and the earlier phrasing that implied it has been removed.

6.3 A Checkpointing Formulation, and What Is Actually Implemented

Where the width k^* exceeds the available memory, some live tensors must be evicted and recomputed instead of stored. The natural formulation of that choice is a constrained optimization problem, which we state here because it makes the trade-off precise — but we state it as a target, not as a description of the shipped scheduler, which is a fixed-interval heuristic (see the end of this subsection).

Let S_v be the memory payload size of node v. We define a binary variable $x_v \in \{0, 1\}$ denoting the checkpointing choice for node v:

$$x_v = \begin{cases} 1 & \text{if } v \text{ is preserved (stored in the static buffer pool)} \\ 0 & \text{if } v \text{ is checkpointed (evicted and recomputed on demand)} \end{cases} \tag{11}$$

Let $T_{\text{fwd}}(v)$ represent the execution latency of the forward rule and $R(v)$ the total operational recomputation overhead incurred when a node is evicted, mapping back to the structural cost of its ancestor paths. The optimization objective seeks to minimize the cumulative latency penalty across the backward pass under a strict physical memory threshold M_{limit}:

$$\min_{x} \sum_{v \in V} (1 - x_v) \cdot R(v) \tag{12}$$

Subject to the formal topological capacity constraints for all execution ticks $t \in [0, t_{\max}]$:

$$\sum_{v \in \text{Live}(t)} x_v \cdot S_v \;\leq\; M_{\text{limit}} \tag{13}$$

Where $\text{Live}(t) = \{v \in V \mid t \in I_v\}$. When $x_v = 0$, the forward rule is executed again during the backward sweep at the point where the pulled dependency structure requires it.

What is implemented. We solve neither the ILP nor its dynamic-programming relaxation. NeuroDSL's `CheckpointSchedule` is a fixed-interval heuristic: parameters and source nodes (which cannot be recomputed) are always retained, and of the remaining nodes one in every k along the topological order is kept as a checkpoint, with k supplied by the caller. Retaining fewer activations only lowers the peak if each one is released as soon as its last consumer has run, so the forward pass interleaves recomputation and release rather than freeing in a single batch at the end; without that interleaving the peak is unchanged, because everything stays resident until after the watermark has already been reached. The formulation above is the target this heuristic approximates, and closing the gap is future work.

6.4 Interactive Visualization

NeuroDSL records an execution trace of the forward and backward passes and exports it as a self-contained HTML viewer (built with Dagre.js and Chart.js). Forward operations are drawn in green, backward operations in red; the viewer allows stepping through individual events and inspecting node values and gradients, with the loss curve displayed alongside. Unlike static model viewers such as Netron [21], the trace is generated by the framework's own dispatcher, so what is displayed is the execution that actually occurred, including the nodes that a partial invalidation caused to be skipped. We use it primarily as a debugging aid for the invalidation logic.

7 Experimental Evaluation

We evaluate NeuroDSL on five fronts: (i) the two localities — invalidation and retrieval — which are the framework's central claim and are measured structurally, (ii) single-step time and peak memory against Flux.jl and PyTorch, (iii) the dynamic optimisations, (iv) end-to-end training, compared against a PyTorch implementation of the same model, and (v) micro-benchmarks to bound the engine's fixed overhead. All experiments were conducted on a laptop equipped with an NVIDIA RTX A5500 (16 GB VRAM) and an Intel Core i9 CPU, running Julia 1.10 and PyTorch 2.5.1.

Because the single-step measurements and the end-to-end measurement point in opposite directions, we state the order of precedence up front: Section 7.8 is the one that reflects what training a model with this framework actually costs, and where they conflict, it should be believed over the micro-benchmarks. Every measurement is reported with the artifact that produced it, and we flag the results that are unfavourable rather than relegating them.

Structural first, wall-clock separately. Throughout this section we apply the discipline set out for `prune_frozen` in Section 4.4: where a claim has a structural form — a count of nodes, of bytes, of traversal steps — we measure that count first, because it is deterministic, involves no timer, and does not depend on the GPU clock or on the machine. Wall-clock figures, where we report them, come with their own protocol and are kept apart from the structural ones rather than summarised into a single number. Each measurement names a negative control, and each cites the artifact that produced it.

7.1 Invalidation Locality: The Traversal Count Does Not See the Graph

The claim of Section 4.5 — that an edit's traversal cost is bounded by its impact subgraph $\mathcal{V}_\theta$ and not by $|\mathcal{V}|$ — is the framework's central claim and the one its title names. Two wall-clock figures cannot establish it, because a time can shrink for reasons that have nothing to do with the traversal. What establishes it is a count, measured while $|\mathcal{V}|$ is made to grow.

The measurement. Three artifacts share the stem `notebook/bench_invalidation_locality`: the benchmark `.jl`, the driver `_driver.sh` that launches it once per (padding, site), and `_results.txt`, which holds every count in Table 5 so a reader can check them without running anything. The benchmark builds a 12-layer `LlamaModel` (dim = 32, 4 heads, hidden 64, seq 8) on CPU, evaluates it so that every node is valid, and then edits one node. The instrument requires no change to the engine: `_invalidate_downstream!` already accepts a `Set{Symbol}` as its fourth positional argument, so the cardinality of that set after the call *is* the number of nodes the traversal touched. No timer is involved, and the figure is independent of the clock and of the machine.

The graph is then padded: for pad $\in \{1, 2, 4\}$ we add pad $- 1$ further chains of `matmul` nodes that are *disjoint* from the model and unreachable from any of its nodes. This inflates $|\mathcal{V}|$ from 601 to 1802 and then to 4204 — a factor of seven — without adding a single node to the downstream cone of anything in the model. If the traversal cost were a function of the graph, that is where it would show.

Table 5: Invalidation locality: nodes visited by `_invalidate_downstream!`, as a function of the edited site (rows) and of the total graph size (columns). One (padding, site) pair per process, fresh graph each time — invalidation mutates the graph, so two measurements in one process would corrupt the second. $|\mathcal{V}|$ grows sevenfold across the columns and no count changes by one unit. Artifacts: `notebook/bench_invalidation_locality.jl`, `notebook/bench_invalidation_locality_driver.sh`, `notebook/bench_invalidation_locality_results.txt`.

Edited site	$\lvert\mathcal{V}\rvert = 601$	$\lvert\mathcal{V}\rvert = 1802$	$\lvert\mathcal{V}\rvert = 4204$
`:input` (negative control)	493	493	493
`layer_1_out`	452	452	452
`layer_6_out`	247	247	247
`layer_12_out` (last)	**1**	**1**	**1**

The mechanism, stated before the numbers.

> **An invalidation visits the forward-reachable set of the edited node and nothing else, so its cost is a property of where the edit is, not of how large the graph around it is.**

Both axes of Table 5 follow from that sentence. Along the rows, the deeper the edited site the smaller its downstream cone, from 493 nodes at the graph input down to a single node when the last block's output is edited — nothing depends on it. Along the columns, padding is unreachable from the model, so it cannot enter any cone, and the count is therefore rigorously constant. The claim being tested is the column direction; the row direction is what makes the quantity non-trivial.

Pre-registered criteria, and their outcomes. Four criteria were fixed before the first launch. *(a) Reproducibility*: the count must be identical across 5 independent process launches — `layer_6_out` returned 247 five times out of five. *(b) Invariance under padding*: a deep site's count must be *rigorously equal* under 4× padding, falsified by an increase of even one unit — it is equal, at every site. *(c) Oracle agreement*: the visited set must match an independently written iterative BFS, built by hand from `rule.inputs` and calling neither `_consumers_index!` nor the function under test, so that the check is not a code agreeing with itself — agreement on **16 of 16** points. *(d) Negative control*: editing `:input` makes the whole model reachable, so locality cannot save anything there and the ratio must be close to 1; a mechanism that looked local even in that case would show that the measurement measures nothing — the control reaches **100.00%** of the invalidatable nodes.

A positive control: making the measure respond. A negative control shows that the measure returns a large value when it should. It does not show that the measure *responds* to the confounder the claim denies matters — a count that were constant by error, through instrumentation that never saw the padding at all, would pass every negative control in Table 5. We therefore added the opposite control, and it is the one that makes the invariance meaningful. In `reachable` padding mode the padding chains consume from `:input`, so they *are* forward-reachable from it, and two exact and opposing predictions were registered before the run: the count at `:input` must grow by exactly $n_{\text{model}} - 1 = 600$ nodes per padding unit, and a deep site's count must not move at all.

Table 6: Positive control: nodes visited when the padding *is* forward-reachable from `:input` (`padmode=reachable`). The control site grows by exactly 600 per padding unit from its unpadded 493, while the three deeper sites are bit-identical to the disjoint case of Table 5. Oracle agreement 8/8. Artifact: `notebook/bench_invalidation_locality_positive_control_results.txt`; the disjoint mode remains the default, so the results of Table 5 are unaffected.

Edited site	pad = 2	pad = 4	Prediction
`:input`	**1093**	**2293**	493 + 600 and 493 + 1800: grows, exactly
`layer_1_out`	452	452	unchanged
`layer_6_out`	247	247	unchanged
`layer_12_out`	1	1	unchanged

Both predictions hold to the unit. The two controls now bracket the measurement from opposite sides: a count that were constant through instrumentation error would fail the positive control, and a count that were really measuring the whole graph would fail the invariance of Table 5. Passing both is what distinguishes locality from an artifact, and no single control could have established it.

What the positive control cost, and what it revealed. The first attempt at it failed: `:input` visited 493 where the oracle said 1093. That was not an engine defect but the `valid`-guard behaviour of Section 4.5 — the reachable padding had never been demanded, so it was already invalid and the traversal correctly declined to re-mark it. Making the padding valid before editing resolved the discrepancy. We record the episode because it is the empirical face of the slack noted in Section 4.5: the traversal really does visit fewer nodes than $|\mathcal{V}_\theta|$ when part of the cone is already invalid, and an oracle computing pure reachability will

therefore over-predict it. The counts reported in this section are all from fully-valid graphs, where the two agree.

Scope of this count. The invariance is exact for what it covers and we do not claim more. It is one architecture at one depth — a 12-layer `LlamaModel` at dim = 32 on CPU — and one padding construction, namely disjoint chains. The four site counts (493, 452, 247, 1) are properties of that graph and would be different numbers in any other; what transfers is the invariance along the columns and the mechanism that produces it, not the counts themselves. We have not counted a topology whose padding is *reachable* from the model, where by construction the count would grow, nor a branched or residual topology at a larger width. A reader wanting the figure for their own model should count it, which the same four-argument call makes possible without modifying the engine.

The control's denominator, and why we report two numbers. Criterion (d) was originally written as $|\text{visited}|/|\mathcal{V}| \geq 0.95$, and against that denominator it measures **82.03**%: as specified, it fails. The reason is structural and was identified and flagged *before* the result was seen. Parameter nodes are graph leaves — no rule produces them — so they can never lie downstream of anything, and no invalidation from any site can ever reach them. The reachable ceiling from the graph input is therefore not $|\mathcal{V}| = 601$ but the invalidatable set, $601 - 108 = 493$ nodes. Against that denominator the control is $493/493 = \mathbf{100.00}$%, exactly saturated. We report both figures and the reason rather than only the favourable one: the criterion as written was wrong about what the ceiling is, and the honest description is that it was mis-specified, not that a threshold was relaxed once the number was known.

7.2 Retrieval Locality

The theorem of Section 4.5 covers *invalidation*. It says nothing about the other half of a reactive step — *retrieval*, the pull that recomputes what an invalidation marked. That half did not have locality: `demand!` scanned the cached topological order from its beginning, so its cost was $\mathcal{O}$(topological position of the target) rather than $\mathcal{O}$(ancestor cone). Locality was proved on one side of the mechanism and absent on the other, and the absence does not follow from the invalidation bound in either direction — a graph can have the one without the other, as this engine did.

We state the novelty plainly, because it is the reason this subsection exists. Retrieval locality is claimed nowhere in the prior work this framework's invalidation bound comes from, and we are not aware of it being reported for a reactive graph engine elsewhere. It is a second locality, on the other half of the mechanism, with its own cost model and its own scope. The asymmetry is closed by `_ancestors_of!` (`src/graph_api.jl`, used by `demand!` in `src/dispatch.jl`); this subsection measures what closing it buys, and on which graphs it buys nothing.

The measurement. Again three artifacts on the stem `notebook/bench_retrieval_locality`: the benchmark `.jl`, the driver `_driver.sh`, and `_results.txt`, which holds every pair in Table 7. The model is the same 12-layer one. Both quantities are structural node counts, timer-free: the *new* cost is the size of the target's real ancestor cone as returned by `_ancestors_of!`; the *old* cost is the target's index in `topo_order!`, which is exactly how many nodes the prefix scan visited before reaching it.

Table 7: Retrieval locality: ancestor cone (new) versus topological prefix (old), in nodes, with their ratio. The cone is invariant in $|\mathcal{V}|$; the prefix is not. Every $|\mathcal{V}| = 601$ entry is exactly 1.00× — see the two conditions stated below the table. Artifacts: `notebook/bench_retrieval_locality.jl`, `notebook/bench_retrieval_locality_driver.sh`, `notebook/bench_retrieval_locality_results.txt`.

Target	$\lvert\mathcal{V}\rvert = 601$	$\lvert\mathcal{V}\rvert = 1802$	$\lvert\mathcal{V}\rvert = 4204$
`:input`	1 / 1 → 1.00×	1 / 2 → 2.00×	1 / 592 → 592×
`layer_1_out`	51 / 51 → 1.00×	51 / 52 → 1.02×	51 / 642 → **12.59**×
`layer_6_out`	301 / 301 → 1.00×	301 / 302 → 1.00×	301 / 892 → **2.96**×
Model output (negative control)	601 / 601 → **1.00**×	601 / 1789 → 2.98×	601 / 4105 → 6.83×

The mechanism, stated before the numbers.

> **A retrieval evaluates the target's ancestor cone and nothing else, so what the fix removes is not work on the target's dependencies but work on subgraphs the target does not depend on at all.**

Two consequences of that sentence have to be stated plainly, because they are conditions and not a general speedup.

First, on a graph holding one model the fix buys nothing. Every entry in the $|\mathcal{V}| = 601$ column of Table 7 is exactly 1.00×, including the shallow targets. In a single chain the topological prefix up to a node and that node's ancestor cone very nearly coincide, so there is nothing to exclude. The gain appears only when the graph contains material the target does not depend on, and it is proportional to how much of that material the old scan would have walked past.

Second, the construction order is part of the measurement and we name it. The padding chains are built *before* the model, so their nodes occupy earlier topological positions — precisely the arrangement the old prefix scan paid for. This is a legitimate configuration, and one the framework's namespaces are designed to allow: a single graph holding several independent subgraphs, of which a caller demands one. It is not, however, the only configuration, and the ratios in the padded columns are ratios for that arrangement, not for an arbitrary graph.

We therefore take `layer_1_out` (12.59×) and `layer_6_out` (2.96×) as the representative figures. The 592× at `:input` is degenerate and we do not headline it: the numerator is a cone of one node, so the ratio measures the size of the padding rather than a property of the mechanism, and dividing by one will produce an arbitrarily large number for an arbitrarily large graph.

Pre-registered criteria, and their outcomes. *(a)* Both quantities identical across 5 independent launches — 51 and 642 five times out of five. *(b)* The new cost of a shallow target rigorously equal under 4× padding while the old cost grows — 51 at every padding, against 51, 52, 642. *(c)* Agreement with an independently written ancestor BFS over `rule.inputs`, calling neither `_ancestors_of!` nor `topo_order!` — **16 of 16**. This oracle is worth one further remark, and we make it as a positive claim rather than an admission: it is the *first independent verification `_ancestors_of!` has ever had*, the function having no occurrence anywhere in `test/`, and it agrees on every point — so the correctness of the cone computation on which `demand!` now depends is established here, not assumed. *(d)* Negative control: the model output depends on almost the whole model, so its cone and the prefix nearly coincide and no gain is possible — measured at 1.000× on the unpadded graph.

The two localities run in opposite directions. It is worth putting the two tables side by side. Invalidation is cheap at *deep* sites and expensive at shallow ones ($493 \to 452 \to 247 \to 1$ as the edit moves toward the output); retrieval is cheap at *shallow* targets and expensive at deep ones ($1 \to 51 \to 301 \to 601$ as the target moves toward the output). Neither mechanism is local everywhere, and each is local exactly where the other is not, so between them they cover the depth of the graph.

7.3 Transformer Block Performance

We compare the forward+backward time on a standard Transformer block [17] (RMSNorm, single-head attention, SwiGLU MLP) with sequence length 64 and varying hidden dimensions, for NeuroDSL versus Flux.jl. Unlike the numbers reported in an earlier version of this table, Table 8 below reflects a range measured across repeated runs on two distinct GPUs (an NVIDIA RTX A5500 Laptop GPU and a desktop RTX 3060 Ti), not a single point-in-time measurement.

Status of this comparison: historical context, not evidence. This is the one measurement in the paper whose per-run output we do not retain. The benchmark lives in `notebook/notebook.ipynb` (`bench_transformer_block`, over `build_transformer_block_neuro` and a Flux mirror), but it was run interactively across roughly fifteen sessions and the individual timings were never written to a log — unlike every result in Section 7.1 through Section 7.8, each of which is driven by a script that archives its raw per-launch lines.

We therefore demote it rather than repair it, and the reason is not only the missing log. This comparison is about *throughput*, which Section 7.8 establishes is not where this framework competes and which the paper does not claim as a contribution. Re-measuring it to the standard of the rest of the evaluation would produce well-evidenced numbers on the one axis we explicitly decline to be judged on, and would invite exactly the reading the rest of the paper argues against. So we retain Table 8 and the counterexample below as historical context — they record what we saw, including a methodological finding about GPU clock behaviour that motivates the pinned-clock protocol used everywhere else in this paper — and we exclude both from Table 20, from the abstract, and from any claim the paper's argument rests on. No conclusion in this paper depends on how NeuroDSL compares with Flux.jl.

Why a range, not a single decimal. Re-running this benchmark repeatedly surfaced two external, non-code sources of variance that a single run masks: (i) GPU dynamic power management can leave the clock well below its boost frequency (as low as 210 MHz against a 2100 MHz ceiling on the laptop GPU) unless the workload sustains enough load to trigger a ramp-up – confirmed directly via `nvidia-smi` clock readings taken before and after warm-up; (ii) background OS/application load adds further jitter, visible as a coefficient of variation exceeding 100% on CPU timings even after discarding the most extreme 10% of samples on each side. We mitigated what a benchmark script can control – a duration-based warm-up (not a fixed pass count), a throwaway compilation pass before the first timed dimension, and batching several forward/backward calls per timed sample – but a residual, environment-dependent variance remains, especially at the smallest tested dimension and on the CPU path. We therefore report the range of medians observed across ∼15 runs rather than a single decimal value.

Table 8: Transformer block, GPU, forward+backward time (ms): NeuroDSL vs Flux.jl, range across repeated runs on two GPUs (RTX A5500 Laptop, RTX 3060 Ti).

dim	Flux (ms)	NeuroDSL (ms)	ND/Flux ratio
256	3.2 – 12.7	2.5 – 10.1	0.53 – 0.85×
512	3.1 – 14.8	2.3 – 10.8	0.69 – 0.87×
1024	3.2 – 13.9	2.4 – 11.0	0.64 – 0.93×

Across every run and both GPUs the ratio stayed below 1, clustering around 0.7–0.85×, with individual runs reaching as low as 0.45×: on this workload NeuroDSL was faster than Flux.jl in every measurement we took, typically by 10–35%, even though the absolute millisecond values swing by up to 4× between runs depending on GPU power state. We record the direction and decline to defend the magnitude; without a retained log neither is a result we would ask a reader to accept.

A workload where the comparison reverses. This result does not generalise across workloads, and we report the counterexample from the same notebook (`notebook/notebook.ipynb`, and subject to the same traceability caveat). On a two-layer dense MLP ($512 \to 1024 \to 512$, batch 64) rather than a Transformer block, NeuroDSL was *slower* than Flux: 15.87 ms vs. 10.43 ms on CPU (1.52×) and 2.71 ms vs. 1.44 ms on GPU (1.88×). The Transformer block is a chain of many medium-sized kernels, where per-node dispatch overhead is amortised; the dense MLP is two large kernels, where it is not. Reporting only the favourable configuration would misrepresent the framework, so we report both and do not claim a general speed advantage over Flux.

Peak GPU memory against PyTorch. `notebook/notebook_py.ipynb` implements a PyTorch mirror of `build_transformer_block_neuro` (same RMSNorm/QKV/attention/SwiGLU architecture, seq = 64, forward + backward + MSE loss) and reports `torch.cuda.max_memory_allocated()`.

Measuring the other side of this comparison correctly is harder than it looks, and an earlier version of this paper got it wrong. That version tracked the NeuroDSL side with `MemTrack`, an instrumentation module that patches the framework's allocation entry points and sums the intercepted allocations. Two problems make the resulting ratio meaningless. First, `MemTrack` sees only what passes through the entry points it patches, and it demonstrably missed some: the CUDA path of `rmsnorm_bwd!` allocated scratch buffers directly, bypassing

the tracker entirely. Second and more seriously, the two quantities are not commensurable. In the PyTorch script, `reset_peak_memory_stats()` is called *after* the model is constructed, so the reported peak includes the resident weights, whereas `MemTrack`'s figure is closer to the allocation activity of one episode. Comparing an episode delta against an absolute peak inflates the result — by a factor of roughly two on this workload — and the range of 2.40×–7.97× reported previously is an artifact of that mismatch, not a measurement. We retract it.

We therefore re-ran the comparison absolute-peak against absolute-peak, instrumenting the NeuroDSL side with the CUDA memory-pool watermark (`CU_MEMPOOL_ATTR_USED_MEM_HIGH`), which is independent of the framework's own allocation paths and so cannot be bypassed by an untracked call. The protocol is: quiesce and synchronise, record the resident baseline, reset the watermark, run one forward + backward episode with a fresh context store, synchronise, read the watermark. The script is `notebook/article_benchmark_vram_probe.jl`; results were identical across 5 trials per size.

Table 9: Peak GPU memory, absolute peak on both sides, single Transformer block (seq = 64, forward + backward + MSE), NVIDIA RTX A5500 Laptop. NeuroDSL figures from the CUDA pool watermark (`notebook/article_benchmark_vram_probe.jl`); PyTorch figures from `torch.cuda.max_memory_allocated()` (`notebook/notebook_py.ipynb`).

dim	NeuroDSL (MB)	PyTorch (MB)	Ratio
256	5.47	20.32	3.71× less
512	18.07	31.38	1.74× less
1024	64.02	74.51	1.16× less

The honest claim is therefore a range of **1.16×–3.71×** less peak memory, and the trend within it matters more than its upper end: the advantage shrinks monotonically as the hidden width grows, and at dim 1024 it is 16%, essentially parity. This is what one would expect if the saving comes from bookkeeping around activation and gradient buffers — a term that grows more slowly than the $\mathcal{O}(d^2)$ weights and $\mathcal{O}(d \cdot \text{seq})$ activations that dominate at larger widths. We have no evidence that the gap persists at production width, and Section 7.8 reports a full training run where it does not.

Two causes, and neither subsumes the other. A reader could conclude from the retraction above that the whole of the difference between the published range and the present one is an instrumentation error. That is one of two causes and we can now separate them, because the engine itself has changed since the submission. We re-ran the same probe against the engine as it stood at commit `b666428` (2026-06-24, the last commit touching `src/` before submission), in an isolated worktree with today's dependencies pinned so that *only* `src/` varied. Table 10 gives both states, measured with the same instrument in the same session.

Table 10: Absolute peak GPU memory on the single-block workload of Table 9, for the engine as submitted and as it stands. Same probe, same session, same PyTorch reference (20.32 / 31.38 / 74.51 MB); the June state was measured in a worktree at `b666428` with dependencies pinned so only `src/` differed. The probe asserts peak equality across 5 trials, so an unstable figure fails rather than being reported. Artifact: `notebook/bench_article_vram_engine_comparison_results.txt`.

Engine state	dim 256	dim 512	dim 1024
As submitted (`b666428`, 2026-06-24)	10.81 MB (1.88× less)	35.44 MB (1.13× **more**)	126.71 MB (1.70× **more**)
Current (2026-08-08)	5.47 MB (3.71× less)	18.07 MB (1.74× less)	64.02 MB (1.16× less)
Peak reduction	−49.4%	−49.0%	−49.5%

The peak halved at every width, and the qualitative position changed: measured honestly, the submitted engine was *losing* to PyTorch at two of the three widths and could claim nothing on this axis. So the two causes are distinct and both real. The published figure was inflated because the instrument on one side summed intercepted allocations while the instrument on the other reported an absolute peak — that error would have existed at any engine state. And the range we report today is not merely that error deflated; it

is a genuine improvement over the engine as submitted, which the corrected instrument would have scored below parity at dim 512 and 1024.

Two limits on how far this should be carried. It is the single-block workload only: the end-to-end training loop of Section 7.8 remains 1.25× worse than PyTorch on peak VRAM, and nothing here changes that figure or softens it. And the June engine's own numbers were never published, so this is not a second false claim being caught — it is the baseline being established, after the fact, so that the present range can be read for what it is.

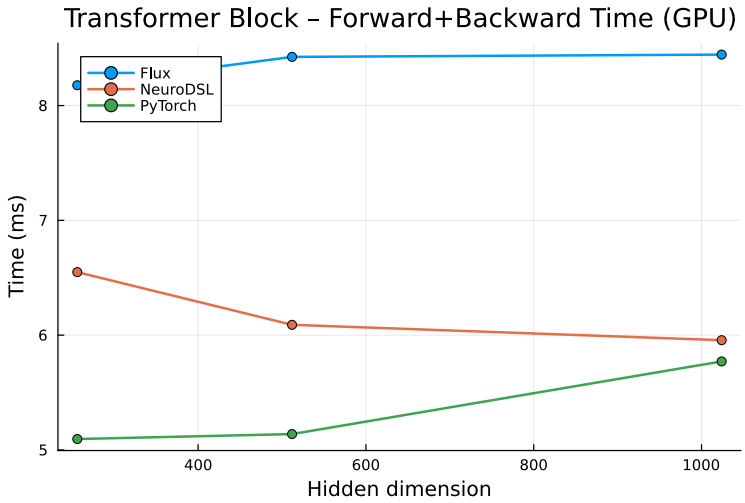

Figure 8: GPU execution time for a Transformer block, NeuroDSL vs. Flux.jl.

7.4 Iris Classification

As an end-to-end sanity check on the backpropagation engine we train a small MLP ($4 \to 16 \to \text{ReLU} \to 3$) on the Iris dataset for 100 epochs with a learning rate of 0.05 and batch size 32 (`notebook/graph_sim.ipynb`). Training loss falls monotonically and the model classifies the 50-sample held-out split without error. We record this only as evidence that the reactive engine drives an ordinary supervised loop to convergence; Iris is separable and a perfect score on a split this small is routine for any working implementation, so it is not a result about NeuroDSL in particular and we do not treat it as one.

Table 11: Iris convergence check (100 epochs, 16 hidden neurons, bias enabled). Test accuracy is on a 50-sample held-out split.

Epoch	Train Loss	Test Accuracy
1	0.4653	–
10	0.1209	–
50	0.0336	–
100	0.0321	**100.0%**

7.5 Micro-benchmarks: Overhead and Memory Scaling

On a tiny model (10×5 matrix multiply + sum, forward and backward timed separately, `notebook/neurodsl_benchmarks.ipy`
NeuroDSL completes the forward pass in a median of **3.8** μ**s** (2.16 KiB allocated) and the backward pass in a median of **3.7** μ**s** (2.12 KiB allocated). At this size the per-node bookkeeping is therefore a few microseconds, not milliseconds; this bounds the fixed overhead of the reactive machinery but says nothing about how it scales, which Section 7.8 addresses directly. We also measured memory allocations for a linear model (Linear → LayerNorm → Linear, where NeuroDSL's `LayerNorm` is implemented as RMSNorm internally) across sizes (see Table 12). Allocations scale linearly with tensor dimensions. Repeated runs of the largest

configuration show no memory leak: after an initial, compilation-affected first call (36,895.0 KB), allocation is exactly reproducible at **30,743.3 KB on every one of the following 9 iterations** – zero drift once warmed up.

Table 12: CPU allocations (KB) for a linear model.

config (M×D)	forward	backward	total
128×256	131.7	2953.5	3085.2
512×512	1027.8	17425.5	18453.3
1024×512	2051.8	28691.5	30743.3

7.6 Dynamic Optimisations

Incremental backward: two quantities, measured apart. Two different things can be meant by "the incremental backward pass saves work", and an earlier version of this paper conflated them under the single word *speedup*. The first is **structural**: how many nodes stop receiving backward treatment. That is a node count. It is deterministic, it is independent of the GPU clock and of the machine, and it is the literal content of the phrase "less recomputation". The second is **wall-clock time**, which additionally depends on how much arithmetic the skipped nodes were carrying and on the state of the hardware while the timer runs. We measure and report them separately below. The artifact is `notebook/bench_prune_frozen.jl`, driven by `notebook/bench_prune_frozen_driver.sh`, with the raw per-launch output preserved in `notebook/bench_prune_frozen_results.txt`; the timing half requires the GPU clock to be pinned (`nvidia-smi -lgc 1402,1402`) and the structural half does not.

The mechanism, stated before the number. By the predicate of Section 4.4, the nodes that still receive backward treatment are exactly those that are trainable or have a trainable ancestor. Hence:

> **The work `prune_frozen` removes is the fraction of the graph lying strictly upstream of the shallowest trainable parameter.**

This is the claim we make, and it is a rule rather than a constant: the resulting percentage is a property of the model and of which of its parameters are frozen, so it is configuration-dependent by construction and a reader should compute it for their own model rather than transfer ours. Counting suffices to do so. Freeze a prefix that reaches the graph input and the entire prefix is removed; leave a single trainable parameter adjacent to the input and nothing upstream of it can be removed, no matter how many parameters are frozen deeper in the network. The two arms measured below are the two extremes of that rule on one topology.

Measured instance (structural). The benchmark builds an 8-layer chain $h_i = \mathrm{relu}(h_{i-1}W_i)$ with dim = 512, 64 rows and an MSE loss, and runs forward + backward on GPU (NVIDIA RTX A5500 Laptop). The graph holds 27 nodes and with the flag off all 27 receive backward treatment. In the **favourable** arrangement — $W_1 \dots W_7$ frozen, only W_8 trainable, so the frozen prefix is contiguous with the graph input — that count falls to 4, i.e. **85.19% of the nodes are pruned**. In the **control** arrangement — W_1 trainable and $W_2 \dots W_8$ frozen, so no part of the graph is upstream of a trainable parameter — it falls to 18, i.e. 33.33% pruned. Both figures were identical in all five launches of their arm, no timer was involved in producing either, and neither depends on the GPU clock.

Wall-clock, as a separate measurement with its own protocol. Timing this mechanism took more care than counting it. A first attempt ran both arms inside one process, with the control arm always second and therefore inheriting the first arm's memory-pool state, and took a single launch per point; it reported +1.0% and then +22.8% for the control arm on two runs of identical code, which is not a measurement. The protocol we report runs *one arm per process*, five independent process launches per arm, 30 clock-alternated repetitions inside each launch, and aggregates by taking the median within a launch and then the min, median and *maximum* across launches, with the GPU clock pinned at 1402 MHz throughout. We report

the maximum rather than a quantile deliberately: at five launches the index of any 90th percentile falls on the last order statistic, so a figure labelled "p90" would be the maximum under a name that suggests a tail estimate the sample cannot support. Under it the favourable arrangement's backward pass is faster by a **median of 69.9%** (min 68.3%, max 76.4%; spread 11.6% of the median), and the control arrangement by a median of 2.79% (min 0.39%, max 4.19%). The separation is complete: no favourable launch fell below 68.3% and no control launch reached above 4.19%, across all ten launches. As an internal consistency check, the two arms' *unpruned* backward times agree — medians 1.102 and 1.091 ms — as they must, since both arms build the same graph and the unpruned pass does the same work in each.

Why a third of the nodes can be worth almost nothing. The control arm carries the more interesting result, and it replaces a bare "no gain otherwise" with a bound: in the unfavourable arrangement the gain is not zero but **bounded at roughly 4%**, and 33.33% of the graph's nodes are still pruned there while yielding no useful time. The composition of what gets pruned explains the discrepancy. In the control arm the nine pruned nodes are all graph leaves — the input, the label, and the seven frozen weight tensors. In the favourable arm, fourteen of the twenty-three pruned nodes are interior `matmul` and `relu` nodes. Node count and arithmetic are not proportional, so the structural percentage is an upper bound on the time that can be saved rather than an estimate of it, and the gap between 33.33% of nodes and $\leq 4.19\%$ of time is that bound being loose. This is also why we report the control arm's time figure as an upper bound and not as a point estimate: it is a ratio of two nearly equal times, so its relative error is large by construction, and its spread across launches is 136% of its own median against 11.6% for the favourable arm.

Exactness: bit-identical on the topology measured. Before either timing arm runs, the benchmark computes the trainable parameter's gradient twice on the same graph state — once with `prune_frozen=false`, once with `true` — and reports the maximum absolute difference between them. Across all ten launches that difference is **exactly** 0 (`max_err=0.000e+00` on every line of `notebook/bench_prune_frozen_results.txt`). On the topologies measured, pruning the frozen sub-DAG is therefore not merely accurate to a tolerance: the two code paths return bit-identical gradients. We state this narrowly and do not generalise it to "always exact". The test suite asserts agreement within 10^{-6} because 10^{-6} is a safe tolerance to assert, not because a discrepancy of that magnitude was ever observed; and Theorem 2 argues exactness in exact arithmetic, which does not imply bit-identity in floating point. Bit-identity is an empirical property of the topologies we measured and is claimed for those only.

Limits of this benchmark. The graph is small — 27 nodes, 8 layers, dim = 512 — and it is one topology (a plain chain, with no branching and no skip connections), on one GPU, at one precision. The structural figures are exact for that configuration and the rule that generates them is general, but the wall-clock figures are not a claim about any other model, and we would not expect the 69.9% to transfer to a topology where the frozen prefix carries a different share of the arithmetic.

Correctness in the test suite. Independently of the benchmark, the framework's test suite runs `backward_graph!` twice on the same graph state — once with `prune_frozen=false`, once with `true` — and compares the trainable parameters' gradients, requiring agreement within 10^{-6} (`test/test_backward_sparse.jl`, `test/test_backward.jl`). This is checked on sequential and branched topologies, including the case where a frozen parameter sits downstream of the trainable one being checked, and on the shared-node configuration where a parameter's gradient is the sum of contributions from two branches — historically the case a pruning bug is most likely to break. All pass.

Scope of the wall-clock claim. The controlled benchmark above is the only wall-clock evidence we report for `prune_frozen`. We had earlier, uncontrolled timings at other depths whose picture was inconclusive, but the measurement artifacts behind them were not retained, so we do not report those figures. The claim is correspondingly narrow: the timing result holds for the configuration it covers — an 8-layer chain, one GPU, clock pinned — and it is not shown to be robust to depth, to branched or residual topologies, or to another machine. The structural figures are unaffected by this, since they follow from a node count rather than from a timer.

Separately, on the 12-block GPT-2-shaped model of Section 7.7 the `sparse=true` path — a different mechanism, which avoids gradient *storage* for frozen parameters without skipping their computation — was 80% slower than a full backward pass while allocating 37% less.

Table 13: Incremental backward. The first block is *structural* — a count of nodes, deterministic and independent of the GPU clock — and is the primary claim; the wall-clock blocks are a separate measurement. Negative percentages are slowdowns. The controlled rows come from `notebook/bench_prune_frozen.jl` driven by `notebook/bench_prune_frozen_driver.sh` (raw per-launch lines in `notebook/bench_prune_frozen_results.txt`), on an 8-layer chain, dim = 512, 64 rows, MSE loss, GPU, **clock pinned at 1402 MHz**; percentages there are aggregated across launches from per-launch ratios, so they need not equal the ratio of the aggregated times shown beside them. The `backward_graph_sparse!` rows are a different mechanism and come from `notebook/GPT2.ipynb` (cells 16–17) and `notebook/neurodsl_benchmarks.ipynb` (cell 13).

Configuration	**Full → pruned/sparse**	**Effect**
Work pruned: nodes receiving backward treatment (structural, deterministic, no timer)		
8-layer chain, $W_1 \dots W_7$ frozen (frozen prefix contiguous with the graph input)	27 → 4 nodes	**85.19% pruned**; identical in 5/5 launches
8-layer chain, W_1 trainable, $W_2 \dots W_8$ frozen (nothing upstream to prune)	27 → 18 nodes	33.33% pruned; identical in 5/5 launches
Wall-clock time, clock pinned, 5 independent launches per arm, 30 alternated reps per launch		
8-layer chain, $W_1 \dots W_7$ frozen	1.102 → 0.324 ms (medians)	**+69.9% median**; min +68.3%, max +76.4%
8-layer chain, $W_2 \dots W_8$ frozen	1.091 → 1.059 ms (medians)	+2.79% median; min +0.39%, max +4.19% (upper bound)
Separate mechanism: `backward_graph_sparse!`, which skips gradient storage *and not computation*		
8-layer MLP, 7/8 frozen, full vs. sparse path, wall-clock	6.02 → 5.78 ms	+4% (within run-to-run spread)
GPT-2-shaped, 10/12 blocks frozen, `sparse=true`, wall-clock	497.1 → 897.0 ms	−80% (slower)
GPT-2-shaped, 10/12 blocks frozen, `sparse=true`, allocations	139,288 → 87,889 KB	−37% allocated
Gradient agreement, pruned vs. unpruned		
8-layer chain, both arms, 10 launches (benchmark)	max abs. difference	**exactly** 0.000e+00 (bit-identical)
Sequential and branched topologies, frozen params up- and downstream (test suite)	agreement $< 10^{-6}$ asserted	all configurations pass

The two mechanisms behave differently and it is worth separating them. `sparse=true` avoids allocating gradient buffers for frozen parameters but still computes their gradients, so it trades a real allocation saving (37%) against added indirection — hence slower but lighter, in every configuration we measured. `prune_frozen` genuinely skips the computation, and whether that pays is decided by the position of the frozen region: with a frozen prefix reaching the graph input it removes 85.19% of the backward nodes and 69.9% of the time, and with a trainable parameter adjacent to the input it removes 33.33% of the nodes and at most 4.19% of the time. Neither result contradicts the impact-subgraph bound of Section 4.5, which bounds a *traversal* and not a wall-clock time; together they show that a node count constrains what can be saved without predicting what is saved. We therefore report the structural figure as the headline and the timing as a separate, protocol-bound measurement, rather than a single number standing for both.

Operator fusion: what it removes, and what it cannot. An earlier version of this paper reported that fusing `matmul+relu` reduced forward time by **37.2%** on CPU, from a single pair of numbers on one

small network. We retract that figure: it is a point measurement in the one regime where the effect is largest, quoted without the width sweep that shows the effect vanishing, and the mechanism does not admit a gain of that magnitude at any realistic size. The mechanism is the place to start.

> **Node fusion calls the vendor GEMM and then applies the elementwise epilogue in a separate pass, so it removes one graph node, one dispatch and one intermediate buffer — never a FLOP, and never a byte of memory traffic.**

This is fusion at the level of the *graph*, not of the *kernel*. Real kernel fusion (cuBLASLt, CUTLASS, Triton) applies the epilogue from registers before the GEMM's output is ever written to memory, and that is what would reduce traffic. NeuroDSL does not do that: `mul!` writes the full output, and the broadcast then reads and rewrites it. Two predictions follow. In *time*, the saving is interpreter overhead, so it must be largest when the tensors are small enough for dispatch to be a material fraction of the total, and must vanish as BLAS comes to dominate. In *memory*, the saving is one resident tensor per fused pattern, which is a node count and therefore does not decay with width at all. The benchmarks below test both, and the memory axis is the one that carries the result.

Correctness first, and it is exact. The artifact is `notebook/bench_fusion.jl` with `notebook/bench_fusion_driver.sh` and `notebook/bench_fusion_results.txt`: a chain of eight `matmul`→`relu` patterns (64 rows, square weights), on which `compile()` applies all eight rewrites, taking the rule set from 16 rules to 8. Every run compares the fused output against the unfused output on identical inputs. The maximum absolute difference is 0.000e+00 — bit-identical — at all six (device, dimension) configurations. Section 5 states semantic preservation as ϵ-equivalence with ϵ unbounded analytically; on this operator, at these six configurations, the measured ϵ is exactly zero. We claim that for what was measured and do not promote it to a general property of the rewrite, for the reason given in the proof of Theorem 3: exactness in exact arithmetic does not imply bit-identity in floating point, and here bit-identity is an empirical fact about a kernel that happens to perform its operations in the same order as the unfused pair.

Table 14: Operator fusion, forward-time gain on a chain of 8 `matmul`→`relu` patterns, 64 rows. One (device, dim) per process, 3 independent launches per point, median reported; GPU clock pinned at 1402 MHz. Positive is faster. On the current engine the gain decreases monotonically with width on both devices, as the mechanism requires. The lower block is the same benchmark and the same driver run against the engine *before* the `_relu_into!` fix, in an isolated worktree with only `dispatch.jl` reverted. Artifacts: `notebook/bench_fusion_results.txt` (current) and `notebook/bench_fusion_prefix_results.txt` (pre-fix), both from `notebook/bench_fusion.jl` under `notebook/bench_fusion_driver.sh`; the medians are that driver's aggregation across launches, so re-running the script alone would not reproduce them.

Device	dim 32	dim 128	dim 512
Current engine (after the `_relu_into!` fix)			
CPU	+11.9%	+2.2%	−0.3%
GPU (clock pinned)	+9.7%	+2.9%	+1.7%
Pre-fix engine, same benchmark and driver			
CPU	**−40.3%**	**−7.9%**	**−22.3%**
GPU (clock pinned)	+7.9%	+6.0%	+2.6%

The shape of Table 14 is the result, not any one of its entries: around 10% at dim = 32, a few percent at 128, and indistinguishable from zero at 512 on both devices. At any realistic width the speed benefit of node fusion is negligible, and we state that plainly rather than quoting the small-tensor corner. No regime comes near 37%, and none could: a transformation that saves a dispatch and leaves the arithmetic and the memory traffic untouched has nothing else to give.

An engine defect found by this benchmark, and worth reporting as a finding. Reaching the upper block of Table 14 took a fix, and the lower block is what the same benchmark measured before it. On

CPU the fused path was not at parity but a *pessimization* — a median of $-\mathbf{7.9}\%$ to $-\mathbf{40.3}\%$ across the three widths, the opposite sign from the mechanism's prediction, which is what made it worth isolating rather than accepting. The separation from the current engine is complete on CPU: at every width the pre-fix and post-fix per-launch ranges are disjoint (for instance $[-55.9, -39.6]$ against $[+11.8, +12.0]$ at dim = 32), so the CPU improvement is not a sampling artifact.

The fix is CPU-only, and the GPU columns should not be read as moving. Their medians differ by at most three points in either direction, the per-launch ranges overlap at two of the three widths, and at dim = 512 where they do not the offset is under one point of median and in the direction of the fixed engine being marginally *slower*. At three launches per point none of that is an effect, and we do not interpret it. It is also what the mechanism predicts: the cost being removed is scalar interpreter overhead in a broadcast loop, and on GPU that epilogue is a kernel launch rather than a scalar loop, so there was nothing there to remove.

The cause, stated without the numbers we cannot source. The cause, found by elimination across the four executable fused operators, is the inline epilogue `output_buffer .= max.(output_buffer, 0f0)`, and what the elimination showed is that neither of its two properties produces the cost alone. It requires their *conjunction*: an aliased broadcast, whose destination is identical to its source, together with a branchy, non-inlinable scalar function — `Base.max` routes through `signbit`/`ifelse`. The two operators that isolate the factors confirm it: `:fused_matmul_add` is aliased but uses `+`, and `:fused_matmul_add_relu` uses `max` but is not aliased, and neither is expensive. The fix is a `@noinline` function barrier, `_relu_into!` in `src/dispatch.jl`. Gradients are unchanged and the package's test suite passes, which any reader can reproduce with `Pkg.test()`.

We give this decomposition qualitatively on purpose. The archived sweep substantiates the *outcome* — the pessimization and its removal — but the per-operator microsecond attribution came from one-off diagnostic scripts that were not retained, so quoting those figures would be citing numbers no file in the repository can produce. That is the standard we applied to the claims we retracted elsewhere in this paper, and we apply it here too. What survives is the conjunction, which is legible in the source, and the sweep, which is archived.

Table 15: Operator fusion, memory. *Activations resident* is structural — a sum of node payload sizes, deterministic, no timer — and falls by the same fraction at every width because it is a node count (17 activation nodes → 9). *Total resident* is the absolute CUDA pool watermark and includes the parameters, which fusion does not touch. Artifacts: `notebook/bench_fusion_memory.jl`, `notebook/bench_fusion_memory_driver.sh`, `notebook/bench_fusion_memory_results.txt`; one arm per process.

	Activations resident	**Total resident VRAM**
dim 32	−47.1%	−37.8% (0.164 → 0.102 MiB)
dim 128	−47.1%	−24.2% (1.031 → 0.781 MiB)
dim 512	−47.1%	**−9.9%** (10.125 → 9.125 MiB)

Memory is the axis this transformation actually has, and the honest figure is the small one. Table 15 reports the same eight-pattern chain instrumented for memory (`notebook/bench_fusion_memory.jl`, launched one arm per process by `notebook/bench_fusion_memory_driver.sh` so that the first arm's buffers cannot inflate the second's baseline, raw lines in `notebook/bench_fusion_memory_results.txt`). The structural quantity is invariant in width, exactly as the mechanism requires: each fused pattern stops materialising the intermediate between its GEMM and its ReLU, so the graph holds 9 activation nodes instead of 17 at every dimension, a 47.1% reduction that involves no timer and no clock.

The figure a reader should carry away is nevertheless the total, and it is the smallest of the three. At dim = 512 the reduction in the framework's actual resident footprint is **9.9**%, and it shrinks with width for a reason that is fully accounted for: the parameters dominate, and fusion does not touch them. The watermark decomposes exactly — at dim = 512 the eight weight matrices occupy 8.000 MiB and the activations 2.125 MiB, summing to the measured 10.125 MiB, and removing 1.000 MiB of activations is 9.9% of that whole. Quoting 47.1% as "memory saved" would be the overclaim this transformation invites: it is the saving

on the one term fusion can reach, expressed as a fraction of that term rather than of the footprint. We therefore lead with the total and use 47.1% only to explain the mechanism.

Scope. These figures are for square weight matrices and 64 rows, where the activation-to-parameter ratio is fixed by the dimension alone. In a real transformer that ratio is set by sequence length and batch size as well, so the activation term — and with it the total reduction — would be a different number, in either direction. We do not extrapolate Table 15 past the configuration measured. The structural 47.1% does transfer, in the narrow sense that it is a count of fused patterns rather than a measurement, and any chain of n such patterns removes n intermediates.

Implementation history on GPU. For completeness we retain the record of how the GPU path reached its present state, since two of its three stages were regressions and the sequence is the reason the final implementation calls cuBLAS instead of a hand-written kernel. An early version of this paper claimed a 53.8% GPU speedup with no reproducible source. Re-measuring exposed a regression of about 4.2×, traced to `_fused_matmul_relu_kernel!` (`src/kernels.jl`) being a hand-written CUDA kernel that reimplemented matmul from scratch, streaming the full reduction dimension from global memory with no reuse, while the unfused path dispatches to cuBLAS via CUDA.jl. Shared-memory tiling narrowed this to 1.9–2.4× slower, but a hand-tiled kernel cannot match cuBLAS's register blocking, tensor cores, and autotuning. The resolution follows the principle GPU libraries use for fused epilogues (cuDNN, cuBLASLt, TensorRT): do not reimplement the matmul — call cuBLAS via `LinearAlgebra.mul!` and fuse only the elementwise epilogue on top. This removed the custom kernel entirely and unified the CPU and GPU code paths, which is why both now show the same width-dependent profile in Table 14.

Table 16: Operator fusion on GPU across three implementation attempts (all measured, none assumed). The final row is superseded by the width sweep of Table 14, which is measured on the current engine.

GPU fusion implementation	**Result vs. unfused (cuBLAS)**
Hand-written untiled kernel (original)	4.2× slower
Hand-written tiled kernel (shared memory)	1.9–2.4× slower
cuBLAS matmul + fused elementwise epilogue (final)	+1.7% to +9.7%, decreasing with width (Table 14)
On CPU, over the same period: −7.9% to −40.3% before `_relu_into!`, −0.3% to +11.9% after	

7.7 Validation on a GPT-2-Shaped Architecture (Toy Scale, CPU)

To validate the dynamic-optimisation mechanisms on a deeper, more realistic topology than the MLPs of the previous section, we build a GPT-2-shaped model with the same block structure as GPT-2 Small (12 Transformer blocks, multi-head attention, learned position embeddings) but at reduced scale for fast iteration: vocabulary size 1000, hidden dimension 256, sequence length 32. Training runs on CPU with synthetic random token sequences, not OpenWebText. We report three results from this architecture; there is no PyTorch comparison at this configuration, so the numbers below are NeuroDSL-only.

Table 17: GPT-2-shaped model (toy scale, CPU) — dynamic optimisation results.

Measurement	**Result**	**Notes**
Forward, incremental fine-tuning (last layer)	166.8 ms → 0.6 ms	99.7% gain
Forward, incremental after an optimizer step	158.7 ms → 29.7 ms	81.3% gain
Backward allocations, full vs. sparse (10/12 layers frozen)	139 288.3 KB → 87 888.9 KB	36.9% less allocated
Backward wall-clock, full vs. sparse (10/12 layers frozen)	497.1 ms → 897.0 ms	**80.4% slower**

Reading these results honestly. The forward-incremental gains are large and consistent with the impact-subgraph bound of Section 4.5, but they are wall-clock figures and a time can fall for reasons unrelated to the traversal, so they are not what establishes the bound — Section 7.1 is, by counting the traversal directly while the graph grows. Read against that count, these two timings are illustrations of it at the deep end

of Table 5: a single-layer fine-tuning step only recomputes a small suffix of the network. The backward-sparse *memory* result is also a genuine reduction (fewer gradient buffers are allocated when most layers are frozen). The backward-sparse *wall-clock* result, however, is a clear counterexample to a naive "sparse is always faster" reading: at this depth and this implementation state, the bookkeeping overhead of partial invalidation outweighs the savings from skipping frozen-layer gradients, so the sparse backward pass is slower in wall-clock time than the full one, despite touching fewer nodes. This is consistent with Section 4.4: correctness of the incremental backward is solidly established, general speed is not, and depends on the specific ratio of overhead to skipped work.

Limitations. This configuration is a toy scale (12 layers, dim 256, synthetic tokens, CPU). Validating the framework at production scale and closing the sparse-backward overhead gap identified above are both left for future work.

7.8 End-to-End Language Model Training: NeuroDSL vs. PyTorch

The benchmarks above are single-step micro-measurements. Because they can hide costs that only accumulate over a long run — and because they are the measurements on which a favourable reading of this framework would rest — we also trained the same model end to end in both frameworks and compared wall-clock time at matched final loss.

The task is character-level language modelling on TinyShakespeare (65-symbol vocabulary): a 4-layer, 4-head Transformer with hidden dimension 256, 2.72M parameters, trained for 20,000 steps on GPU. The two implementations were written independently against the same architecture and hyperparameters; the correctness cross-check is that they converge to the same place from the same starting loss, which they do. Results are in Table 18 (`notebook/real_llm_neurodsl_results.json`, `notebook/real_llm_pytorch_results.json`).

Table 18: End-to-end character-level LM training, 20,000 steps, RTX A5500 Laptop. Both runs reach the same validation loss; NeuroDSL takes 2.23× longer to get there. The peak-VRAM row is the maximum over the whole loop and is the quantity comparable to PyTorch's; it is resolved by episode in Table 19, where its sign is not uniform.

Measurement	NeuroDSL	PyTorch	Ratio
Time per step	25.29 ms	11.35 ms	**2.23× slower**
Total wall-clock (20,000 steps)	505.7 s	227.0 s	2.23× slower
Peak VRAM	91.8 MB	73.6 MB	**1.25× more**
Final validation loss (nats/char)	1.623	1.620	matched
Initial loss (nats/char)	4.19	4.29	—

Peak memory, resolved by episode. The single peak-VRAM figure in Table 18 is the maximum over the whole loop, which is the quantity comparable to `torch.cuda.max_memory_allocated()` but which hides that the loop contains three structurally different episodes. We therefore re-measured them separately on the current engine, using the CUDA pool watermark (`CU_MEMPOOL_ATTR_USED_MEM_HIGH`) in an isolated process, and the earlier figures reproduce exactly. The script is `notebook/real_llm_vram_probe.jl` and its per-episode output is preserved in `notebook/real_llm_vram_probe_results.txt`, against the same PyTorch reference of **73.62 MB** (`notebook/real_llm_pytorch_results.json`).

Table 19: Peak VRAM by episode, char-LM (dim 256, 4 layers, 4 heads, 2 722 369 parameters), against a PyTorch reference of 73.62 MB measured over the whole loop. Ratios above 1 are unfavourable to NeuroDSL. Artifact: `notebook/real_llm_vram_probe.jl`, output archived in `notebook/real_llm_vram_probe_results.txt`.

Episode	NeuroDSL	Ratio vs. PyTorch
`train_step` (default)	91.78 MB	**1.25× worse**
`train_step` (`release_values=true`, opt-in)	80.27 MB	1.09× worse
`val_window` (forward only)	51.44 MB	**0.70×**
`gen_token` (autoregressive)	51.80 MB	**0.70×**

The probe also re-reads the resident baseline after 25, 50, 75 and 100 further cumulative training steps, and it does not grow: the training figure reflects simultaneous residency rather than an accumulation. We report that as a check performed and not as a measured value, because the probe prints those four readings to the console without archiving them — the four peaks in Table 19 are the archived quantities.

What this changes. Two things, and both are corrections to claims a reader would otherwise draw from the micro-benchmarks.

First, NeuroDSL is not competitive with PyTorch on throughput. At 2.23× slower per step, with the loss curves lying on top of each other, the gap is not a measurement artifact and it is not close. The favourable Transformer-block comparison of Section 7.3 was against Flux.jl, not PyTorch, and it does not transfer.

Second, and this is the point the single number obscured, the memory comparison does not have one sign. It splits by whether the episode has a backward pass. NeuroDSL uses **30% less** peak VRAM than PyTorch on the forward-only episodes — validation and autoregressive generation, both at 0.70× — and **25% more** on the training step. A sentence claiming that NeuroDSL uses less memory than PyTorch would therefore be false, and so would its negation.

The trade-off, stated as one. What produces both signs is a single property, and it is the same property Results Section 7.1 and Section 7.2 rest on:

> **The persistent graph keeps activations resident. That residency is what makes an edit local — an invalidation can mark a cone and a retrieval can recompute it precisely because everything outside the cone is still there — and it is also what makes the training step heavier, because a backward pass adds gradient and optimizer state on top of activations that were never released.**

Read that way the episode split is not two results but one mechanism seen from two sides. On a forward pass, residency costs nothing extra and the engine's tighter buffer reuse shows through as a 30% advantage. On a training step, residency is additive with everything backpropagation needs, and the same mechanism costs 25%. NeuroDSL trades memory residency for locality; it does not reduce memory, and we do not claim it does.

Recovering the training gap, as an opt-in. `backward_graph!` accepts `release_values=true`, which releases activation values as the reverse pass consumes them. It recovers most of the training-step gap — from 1.25× to 1.09× — at a measured cost of roughly 10% in time, and it leaves the graph with activations missing, so a full `demand!` is required before the next step. That is exactly why it is not the default: the mode gives up part of the residency that the locality results depend on, in exchange for a footprint closer to PyTorch's. We present it as a trade the caller can make, not as the framework's normal operating point.

What the framework does provide is what Section 4.2, Section 7.1 and Section 7.2 describe: edits and retrievals whose cost is bounded by a cone rather than by the graph, and gradients that remain correct when only part of the graph has been re-evaluated. Those are properties of the execution model, and they are not purchased with throughput — they cost it.

7.9 Function Approximation ($\sin(2\pi x)$)

As a second convergence check on a non-linear regression target, we train a two-layer tanh MLP ($1 \to 64 \to 64 \to 1$) to approximate $\sin(2\pi x)$ on $[0, 1]$. After 500 epochs the fit tracks the target closely (Figure 9). Like the Iris check, this verifies that the engine trains; it is not a comparative result.

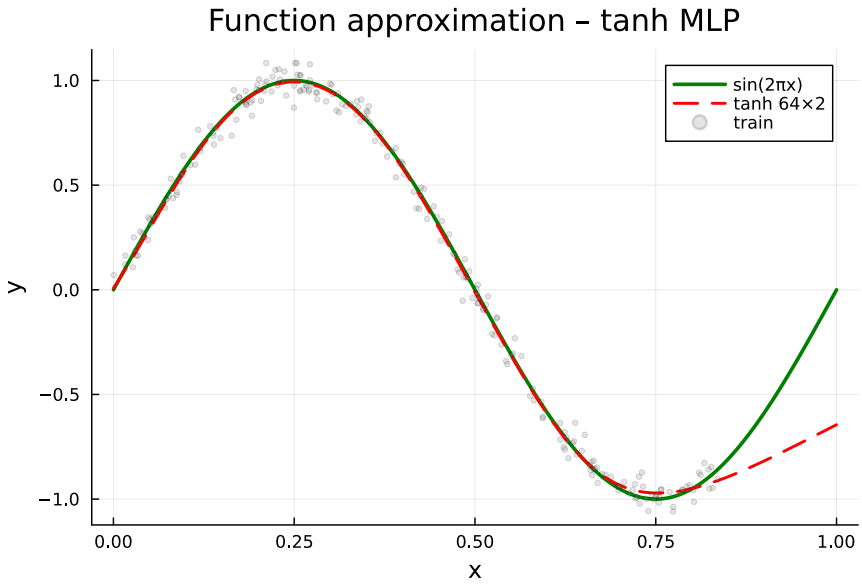

Figure 9: Approximation of $\sin(2\pi x)$ with a tanh MLP.

7.10 Summary of Results

Table 20 consolidates the experimental results, favourable and unfavourable together. The pattern worth noting is that what supports NeuroDSL is structural — counts of traversed nodes, of resident tensors, of pruned backward nodes, all deterministic and clock-independent — while what goes against it is wall-clock throughput and the memory cost of a training step. The rows are grouped accordingly, structural first.

8 Related Work

Static frameworks (PyTorch, JAX, TensorFlow) trace or define the graph once and then execute it repeatedly [2, 3, 4]. They are mature and efficient, but do not allow architectural changes during training without complex workarounds. *Dynamic graph systems* such as DyNet [7] and Chainer [8] offered some flexibility in the past, but they were slow on GPU and are no longer maintained. Modern eager-mode PyTorch is dynamic in the sense that the graph is rebuilt on every forward pass, but it does not persist a mutable data structure that can be incrementally updated.

Memory optimisation techniques like gradient checkpointing [9], ZeRO [10], or activation offloading [11] reduce peak memory, and they operate at scales NeuroDSL has not been tested at. NeuroDSL's interval-colouring planner solves a narrower and much easier problem — the minimum buffer-slot count for a fixed execution order, a classical interval-graph result [12] — and its distinguishing property is that the resulting assignment is exposed as an inspectable object on a graph that can still change, not that it dominates these techniques.

Operator fusion has been explored by compilers such as TVM [18] and XLA, both of which generate code for fused patterns. NeuroDSL performs fusion as a graph-level rewrite with no compilation step, which is why it is limited to patterns for which a kernel already exists. *Category-theoretic deep learning* (Catlab.jl [13], Catgrad [14]) provides a rigorous foundation for automatic differentiation; NeuroDSL does not build on

that foundation, and we mention these projects to situate the design space rather than to claim a relationship to them.

Visualization tools like TensorBoard [20] and Netron [21] offer static or aggregated views; NeuroDSL replays a recorded step-by-step trace of both forward and backward passes. *Domain-specific languages for ML* such as Halide [22] and Dex [23] propose new programming models for expressing differentiable computations. NeuroDSL's Rule DSL differs in that it operates at the *graph level* rather than the kernel or type-system level, and is designed for runtime mutability rather than ahead-of-time compilation.

9 Limitations and Future Work

Throughput. The clearest limitation is the one measured in Section 7.8: at 2.23× PyTorch's time per step on a real training run, NeuroDSL is not a framework one would choose to train a model quickly. The cost is structural rather than incidental — every node carries validity state, and evaluation is a pull through a symbol-keyed dispatch table rather than a fused, compiled kernel sequence — so closing the gap is not a matter of tuning.

Memory residency. NeuroDSL cannot be described as using less memory than PyTorch, nor as using more: it uses 30% less on forward-only episodes and 25% more on a training step (Table 19), and the reason for both is one mechanism — the persistent graph keeps activations resident. That residency is what the locality results of Section 7.1 and Section 7.2 are built on, since a cone can be marked and recomputed precisely because what surrounds it is still there. It is therefore a design decision with a measured price on one episode type and a measured benefit on another, not a defect awaiting repair. The genuine limitation is narrower: we do not have a design that recovers the training-step footprint without giving up residency. The `release_values=true` mode trades some of it back at roughly 10% in time, which is why it is opt-in rather than default, but it is a trade and not a solution.

Fusion, in its actual scope. Node fusion removes a graph node, a dispatch and an intermediate buffer. It does not remove a FLOP and it does not reduce memory traffic, because the vendor GEMM writes its full output before the elementwise epilogue reads it back. The consequences are both measured and both narrow: the forward-time gain falls from about 10% at dim = 32 to roughly zero at dim = 512 (Table 14), and the resident-footprint reduction is 9.9% at dim = 512 (Table 15), the invariant 47.1% figure being a reduction in activation nodes rather than in the footprint. Reaching a regime where fusion matters for either quantity requires kernel-level fusion — an epilogue applied from registers before the GEMM output is written — which NeuroDSL does not implement. The measurements above are for square weights and 64 rows; at a real transformer's activation-to-parameter ratio the total figure would differ, and we do not extrapolate it.

Fusion coverage. Two entries exist in `FUSION_TABLE` and only one of them, `matmul+relu`, has a kernel; separately, seventeen distinct fused operators are named across the rewrite-rule sets and four of them have an execution branch, the remainder being matched and then refused by the dispatch guard. Coverage is bounded by the kernel library, so extending it means writing kernels, not patterns. An automated pattern-mining pass is of limited use until that changes.

Scope of the locality measurements. The two locality results are exact for what they count and narrow in what they cover. The invalidation benchmark instruments the *forward* sweep, `_invalidate_downstream!`; the gradient-nullifying sweep `_invalidate_upstream!` is not counted there. The retrieval benchmark's ratios are ratios for a graph in which independent subgraphs were constructed *before* the model and therefore occupy earlier topological positions — a configuration the framework permits and one the old prefix scan penalised, but a stated condition of the measurement and not a property of arbitrary graphs. On a graph holding a single model the retrieval fix is worth exactly 1.00×. Both benchmarks run on one topology (a 12-layer `LlamaModel`, dim = 32, CPU); the counts are exact for it and the mechanism that generates them is general, but we have not counted a branched or multi-model topology beyond the padded configuration described.

Distributed training. There is no multi-device support in the framework. A two-GPU data-parallel experiment with periodic weight synchronisation exists as a notebook (`notebook/MNIST_par.ipynb`) but is not part of NeuroDSL itself, and there is no multi-node path. Bringing the framework to the level of an established data-parallel implementation [19] is out of scope for this work.

Checkpoint scheduling. The ILP formulation of Section 6 is not solved; the shipped scheduler retains every k-th recomputable node along the topological order.

The memory planner does not reduce memory. This is the flattest way to put Section 6.2 and we prefer it to a hedge. Interval coloring is optimal in slot count, but on a graph expecting a backward pass no two activation lifetimes are disjoint, so the optimum is the node count and no slot is shared — measured as 201 slots for 201 nodes. A forward-only plan does share (201 $\rightarrow$ 40, and 0.20–0.40 of the node count across four graphs), and after three fixes the planned path is bit-exact with a 92.1% pool hit rate where it previously reused nothing and failed on a second invocation. Even so, its peak VRAM is 46.96 MB, identical to consulting no plan at all, because `release!` returns buffers to a pool bucket rather than to the device and a watermark does not fall when nothing is freed. What reduces the forward-only peak is `demand_release!` at 16.96 MB. Two things would be needed to make the planner a memory mechanism: a pool that can return memory to the device under a high-water policy, and a byte-aware coloring, since slots dimensioned for their largest tenant do not minimise bytes. Neither exists. The claim we retain for the planner is inspectability, not saving.

Compilation integration. NeuroDSL does not integrate with XLA or LLVM for ahead-of-time kernel generation. A hybrid approach — keeping the mutable DAG at the framework level while delegating hot sub-graphs to a compiled backend — is the most plausible route to reducing the throughput gap, and is untried.

Gradient correctness and pruning at scale. `prune_frozen`'s gradients have been checked against the unpruned path within 10^{-6} on sequential and branched topologies in the test suite, and measured bit-identical (difference exactly 0) on the 8-layer GPU chain of Table 13. Neither is a large-Transformer or mixed-precision validation, and we claim bit-identity only for the topologies on which we measured it. Theorem 2 gives the argument for exactness in exact arithmetic; it does not substitute for that validation. The pruning figures inherit the same limit from the other direction: 85.19% is exact for a 27-node chain with a frozen prefix, and the general statement — that the pruned work is the part of the graph upstream of the shallowest trainable parameter — has been verified structurally on that chain only, not on a branched or residual topology where the shallowest trainable parameter may not separate the graph so cleanly.

Scale of evaluation. Every measurement in this paper is at or below 2.72M parameters. Nothing here supports a claim about behaviour at production scale, and the one trend we can extrapolate — the memory ratio in Table 9 — points the wrong way.

10 Conclusion

We have presented NeuroDSL, a deep learning framework centred on a computational graph that is persistent and mutable rather than rebuilt each step. Its contribution is a set of properties of the execution model, and the two central ones are now measured rather than argued. A change to a parameter or to the topology traverses only its impact subgraph: inflating the total graph sevenfold with disjoint unreachable branches leaves the traversal count identical at every edited site, in agreement with an independent oracle on 16 of 16 points, with the negative control saturating the invalidatable set. A retrieval evaluates only the target's ancestor cone: that half of the mechanism had no such bound before — evaluation scanned the cached topological order from its start — and closing it makes the retrieval cost invariant in total graph size, worth 12.59$\times$ and 2.96$\times$ on shallow and mid-depth targets in a graph holding independent subgraphs and exactly 1.00$\times$ in one that does not. The two run in opposite directions along depth, invalidation cheap at the deep end and retrieval at the shallow end, which is why the pair rather than either one is the claim. Alongside

them: the backward pass remains correct when only part of the graph has been re-evaluated, checked within 10^{-6} and measured bit-identical on the topology instrumented directly; it can skip the part of the graph upstream of the shallowest trainable parameter, 85.19% of the backward nodes on a frozen-prefix configuration; the node-fusion rewrite's output is bit-identical to the unfused composition at every configuration tested; and the buffer assignment is an inspectable object rather than an internal allocator decision — inspectable being the whole of the claim, since we measure that the plan shares no slot on a training graph and changes the peak by nothing even when it does share.

It is worth being explicit about what this does not buy. NeuroDSL is about 2.23× slower per step than PyTorch on an end-to-end language-model training run at matched validation loss. On memory the honest statement is not a comparison but a trade, and it is a designed one: 30% below PyTorch on forward-only episodes and 25% above it on the training step, both consequences of the single decision to keep activations resident — the decision that makes the two localities possible in the first place. A claim that the framework uses less memory would be false; so would a claim that it uses more. It buys locality with residency, deliberately, and the episode split is what that purchase looks like when measured. Its node fusion saves a dispatch rather than a FLOP, so its speed benefit is around 10% only where the tensors are small enough for dispatch to matter and is negligible at realistic width. And the mutable graph does not make any training procedure newly *possible*: a static framework can rebuild a model and copy weights across. What changes is that the edit becomes local and in-place, preserving state outside the affected region instead of reconstructing everything — a difference in cost and continuity, which is the property the framework is built to provide and the one on which it should be judged.

The natural next steps follow from the limitations above rather than from these results: counting the two localities on branched and multi-model topologies beyond the padded configuration measured here, kernels to widen fusion coverage, delegation of hot sub-graphs to a compiled backend to reduce the throughput gap, and validation at a scale where the memory trend can be tested rather than extrapolated.

NeuroDSL is open-source and available at `https://github.com/nevermind78/NeuroDSL`. A demonstration video is included in the repository.

References

[1] Julius Technology, *JuliusGraph: A Dynamic Graph Engine*, `https://juliustechco.github.io/JuliusGraph/dev/`, 2024.

[2] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.

[3] J. Bradbury et al., *JAX: composable transformations of Python+NumPy programs*, GitHub 2018.

[4] M. Abadi et al., *TensorFlow: A System for Large-Scale Machine Learning*, OSDI 2016.

[5] M. Innes et al., *Flux: Elegant Machine Learning with Julia*, JOSS 2018.

[6] M. Innes, *Don't Unroll Adjoint: Differentiating SSA-Form Programs*, arXiv 2018.

[7] G. Neubig et al., *DyNet: The Dynamic Neural Network Toolkit*, arXiv 2017.

[8] S. Tokui et al., *Chainer: a Next-Generation Open Source Framework for Deep Learning*, NIPS 2015.

[9] T. Chen et al., *Training Deep Nets with Sublinear Memory Cost*, arXiv 2016.

[10] S. Rajbhandari et al., *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*, SC 2020.

[11] M. Rhu et al., *vDNN: Virtualized Deep Neural Networks*, ASPLOS 2016.

[12] R.P. Dilworth, *A Decomposition Theorem for Partially Ordered Sets*, Annals of Mathematics 1950.

[13] E. Patterson et al., *Catlab.jl: A Framework for Applied Category Theory in Julia*, ACT 2021.

[14] P. Wilson et al., *Categorical Foundations of Gradient-Based Learning*, arXiv 2022.

[15] B. Zhang and R. Sennrich, *Root Mean Square Layer Normalization*, NeurIPS 2019.

[16] N. Shazeer, *GLU Variants Improve Transformer*, arXiv 2020.

[17] A. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.

[18] T. Chen et al., *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*, OSDI 2018.

[19] S. Li et al., *PyTorch Distributed: Experiences on Accelerating Data Parallel Training*, VLDB 2020.

[20] M. Abadi et al., *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*, 2015 (TensorBoard).

[21] L. Roeder, *Netron: Visualizer for Neural Network, Deep Learning and Machine Learning Models*, GitHub 2018.

[22] J. Ragan-Kelley et al., *Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines*, PLDI 2013.

[23] A. Maclaurin et al., *Dex: Array Programming with Typed Indices*, arXiv 2019.

Table 20: Summary of experimental results. Rows marked ↓ are unfavourable to NeuroDSL. *Structural* rows are node or byte counts: deterministic, timer-free, independent of the GPU clock. Every row here is backed by a retained artifact; the Flux.jl comparison of Section 7.3, which is not, is deliberately excluded.

Measurement	**Result**	**Comparison / scope**
	Locality (structural: traversal counts, no timer)	
Invalidation traversal, $\|\mathcal{V}\|$ grown 7×	493 / 452 / 247 / 1 nodes	identical at every $\|\mathcal{V}\|$; oracle 16/16; negative control saturated at 100.00% of invalidatable nodes, Table 5
Invalidation, positive control (reachable padding)	493 → 1093 → 2293 nodes	count grows by exactly 600/unit at `:input` while deep sites are unchanged; oracle 8/8, Table 6
Retrieval, ancestor cone vs. topological prefix	12.59× / 2.96×	shallow / mid target under 4× padding; **exactly 1.00× on a single-model graph**, Table 7
	Throughput and memory	
Peak GPU memory, single block	5.5–64.0 MB	1.16–3.71× less than PyTorch eager; gap closes as width grows, Table 9
↓ Wall-clock per step, 20k-step LM run	25.29 ms	2.23× *slower* than PyTorch at matched loss, Table 18
↓ Peak VRAM, LM `train_step`	91.8 MB	1.25× *more* than PyTorch; 1.09× with opt-in `release_values` at ∼10% time cost, Table 19
Peak VRAM, LM `val_window` / `gen_token`	51.4 / 51.8 MB	0.70× PyTorch (30% less) on forward-only episodes; no leak over 100 further steps, Table 19
Fusion, forward time, dim 32 / 128 / 512	+9.7–11.9% / +2.2–2.9% / −0.3 to +1.7%	CPU and GPU alike; saves a dispatch, never a FLOP, so negligible at realistic width, Table 14
↓ Memory planner, slots shared (default path)	201 of 201 nodes	*zero* sharing: with a backward pass no lifetimes are disjoint, so the Dilworth optimum is the node count, Table 3
Memory planner, forward-only plan	201 → 40 slots	0.20–0.40 of node count across 4 graphs; 92.1% pool hit rate, bit-exact, Table 3
↓ Memory planner, peak VRAM	46.96 MB	*identical* to no plan; pooling is not freeing. `demand_release!` is 2.77× lower at 16.96 MB, Table 4
Fusion, memory (structural / total)	−47.1% / −9.9%	17 → 9 activation nodes at every width; total at dim 512, parameters dominate and are untouched, Table 15
Fusion, numerical agreement	exactly 0	fused vs. unfused output, bit-identical at all six (device, dim) points, before and after the fix, Table 14
Fusion, CPU path before `_relu_into!`	−7.9% to −40.3%	a pessimization, removed; pre- and post-fix launch ranges disjoint at every width; GPU unaffected, Table 14
Incremental backward, gradient agreement	exactly 0	pruned vs. unpruned, bit-identical over 10 launches; test suite asserts $< 10^{-6}$, Table 13
Incremental backward, work pruned (structural)	85.19% / 33.33% of nodes	frozen prefix reaching the input / trainable param at the input, Table 13
Incremental backward, wall-clock (pinned clock)	+69.9% / $\leq$ 4.19%	same two arms, 5 launches each, 8-layer chain only; depth-robustness not shown, Table 13
↓ Backward-sparse path, wall-clock	−80%	GPT-2-shaped, 10/12 frozen; slower but −37% allocated, Table 13
GPT-2-shaped, forward-incremental (last layer)	166.8 → 0.6 ms	99.7%; toy scale, CPU, Section 7.7
Iris convergence check	no test errors	50-sample split; sanity check only, Table 11
Micro-benchmark overhead (tiny graph)	3.8 / 3.7 μs	forward / backward median, Section 7